

Pipelining: Basic and Intermediate Concepts

Professor Hugh C. Lauer

CS-4515, System Programming Concepts

(Slides include copyright materials from Computer Architecture: A Quantitative Approach, 5th ed., by Hennessy and Patterson and from Computer Organization and Design, 4th ed. by Patterson and Hennessy)

From OS Course

- **Three models of parallelism in computing...**

- Data parallelism
- Task parallelism
- Pipelining

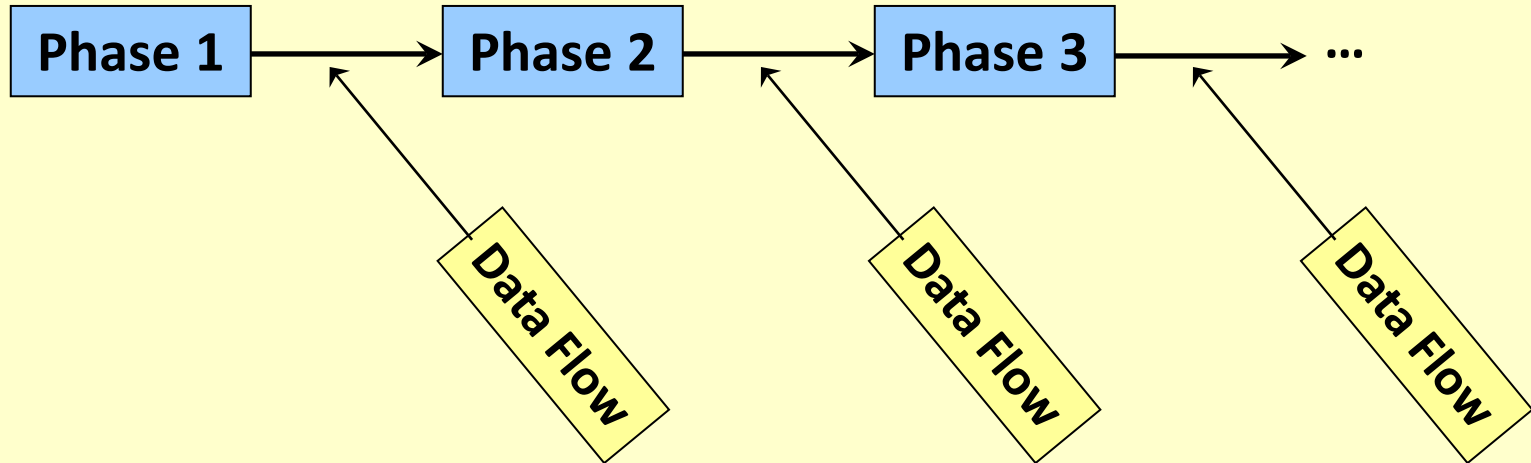
- **... plus one new one**

- Google – massive parallelism (warehouse scale)

From OS Course

- **Three models of parallelism in computing...**
 - Data parallelism
 - Task parallelism
 - Pipelining
- ... plus one new one
 - Google – massive parallelism (warehouse scale)

Pipelined Parallelism



- **Assume phases do not share resources**
 - Except data flow between them
- **Phases can execute in separate threads in parallel**
 - I.e., *Phase 3* works on item i , while *Phase 2* works on item $i+1$, while *Phase 1* works on item $i+2$, etc.

Computer Architecture

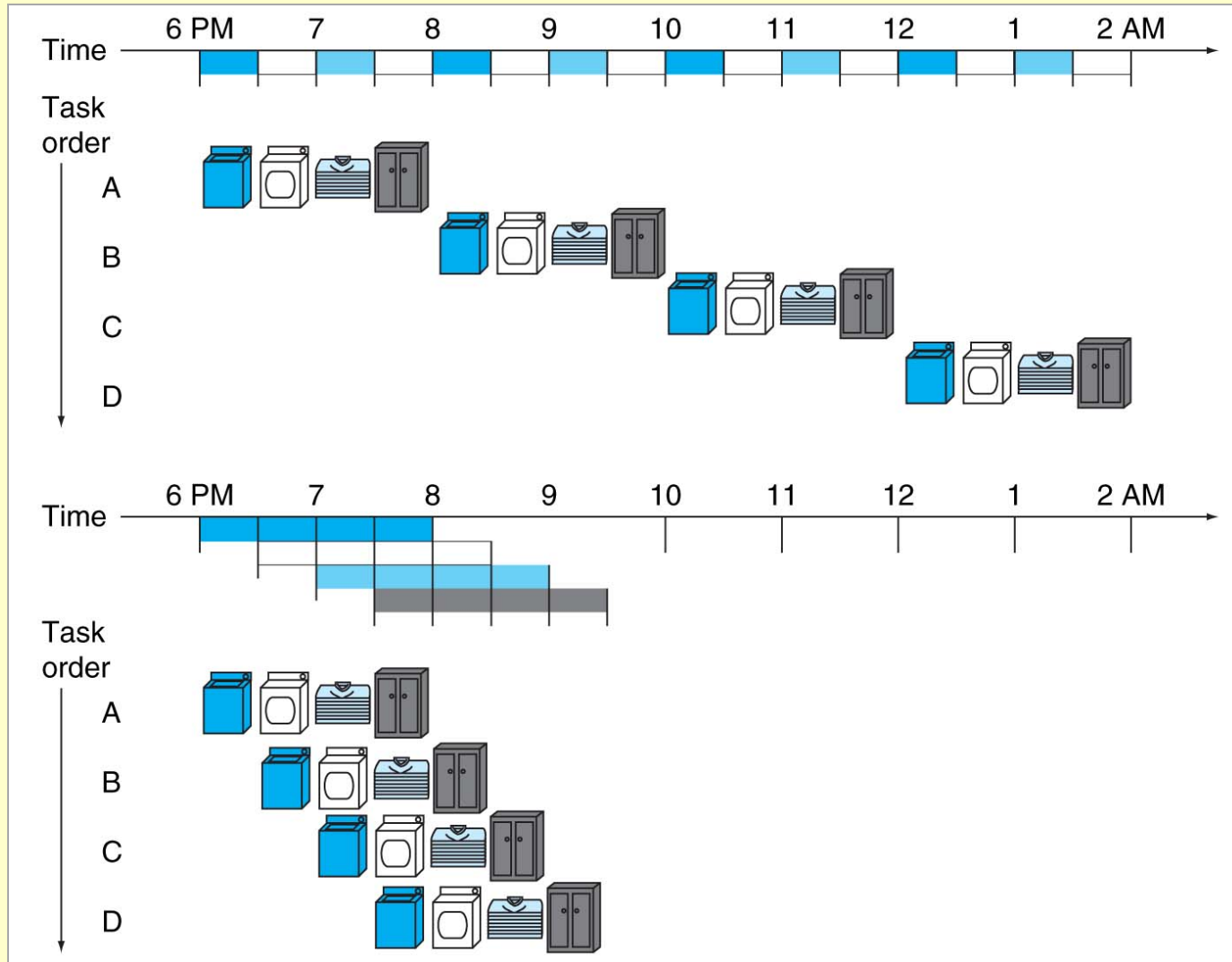
- Same model applies to executing instructions within a processor
 - CPUs
 - Graphics
- (Nearly) all modern processors *pipelined* at instruction level since mid-1980's
 - Some specialized computers were pipelined long before
- RISC instruction sets designed specifically to accommodate pipelining
 - And to work hand-in-hand with modern compilers

Reading Assignment

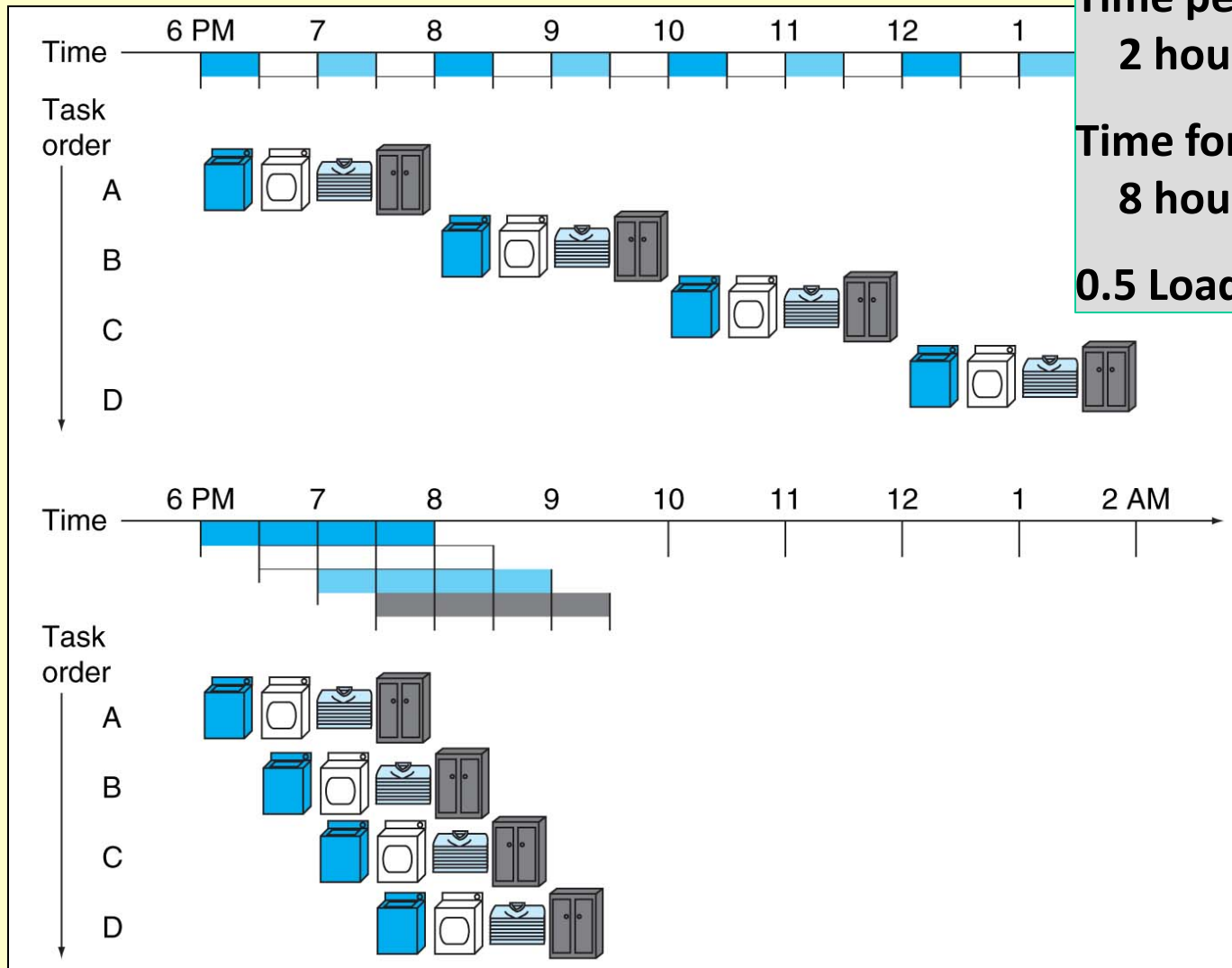
- **Appendix C – *Pipelining: Basic and Intermediate Concepts***
 - Especially §C.4-C.6
- **Also re-read/review §1.8 of Chapter 1, especially *Processor Performance Equation*, pp. 49–51**
 - Understand what is meant by *CPI*
 - I.e., Cycles per Instruction

Pipelining is also discussed in
Byrant & O'Hallaron, §§4.6–4.5

Clothes-washing Analogy



Clothes-washing Analogy (continued)

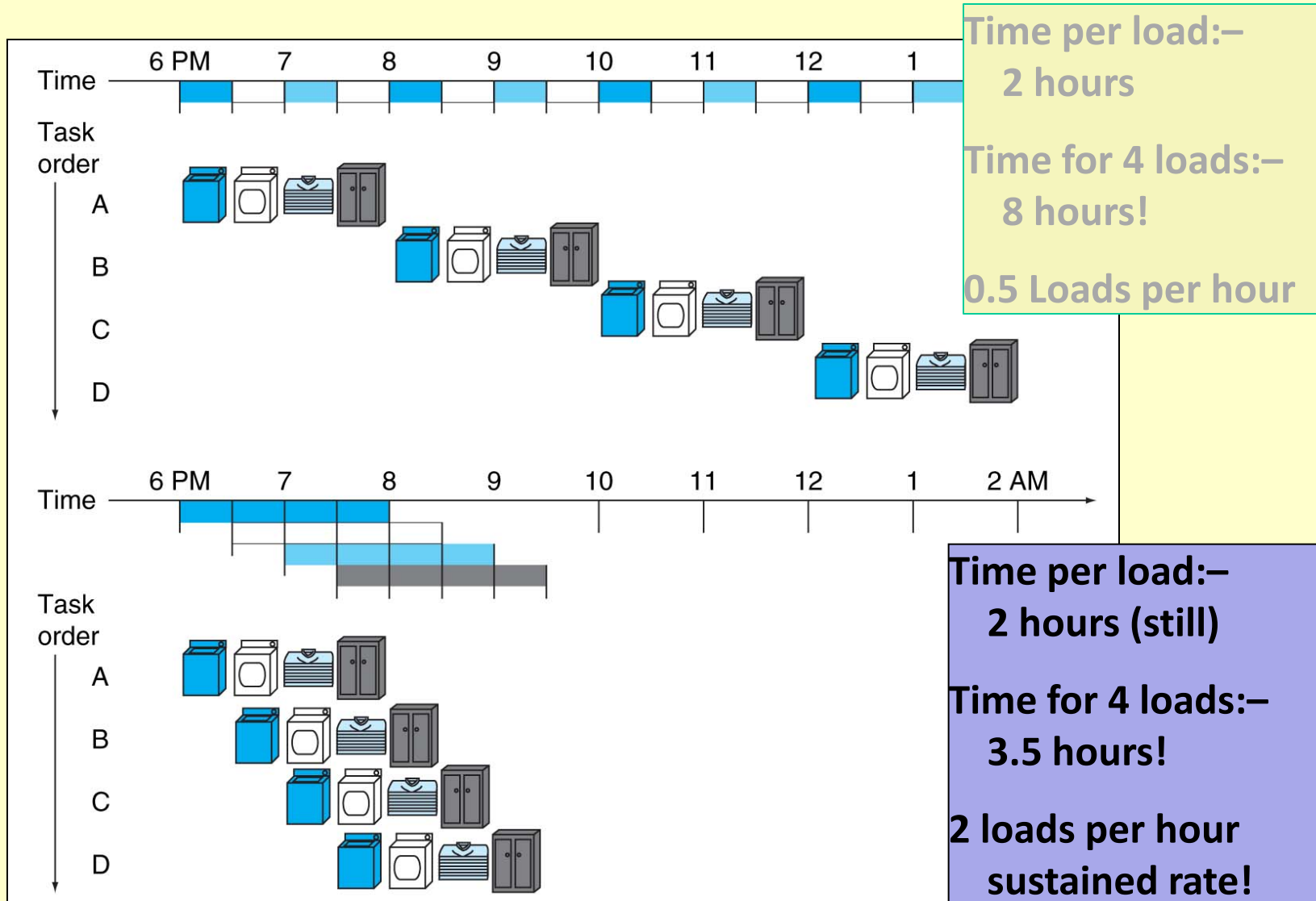


Time per load:—
2 hours

Time for 4 loads:—
8 hours!

0.5 Loads per hour

Clothes-washing Analogy (continued)



Better Analogy

- **Production line for automobiles or other manufactured products**
- **Figure of merit:–**
 - Number of cars per day off the line

Pipeline in Computer Processor

- Partition instruction execution into *stages*
 - I.e., *phases* on the OS slide
- Each *stage* uses a different set of hardware resources
 - Avoid conflicts
- *Stages* can execute concurrently, pipeline style

Example — MIPS Pipeline

■ Instruction fetch

- Increment instruction counter as part of this stage

■ Instruction decode and Register fetch

- I.e., read operand(s) from register(s)

■ Execution *or* Address Calculation

- Arithmetic operation using fetched registers or immediate operand

■ Memory access

- Read from or write to (main) memory

■ Write-back to Register

- Store result into register again

Example — MIPS Pipeline

■ Instruction fetch

- Increment instruction counter as part of this stage

■ Instruction decode and Register fetch

- I.e., read operand(s) from register(s)

■ Execution *or* Address Calculation

- Arithmetic operation using fetched registers or immediate operand

■ Memory access

- Read from or write to (main) memory

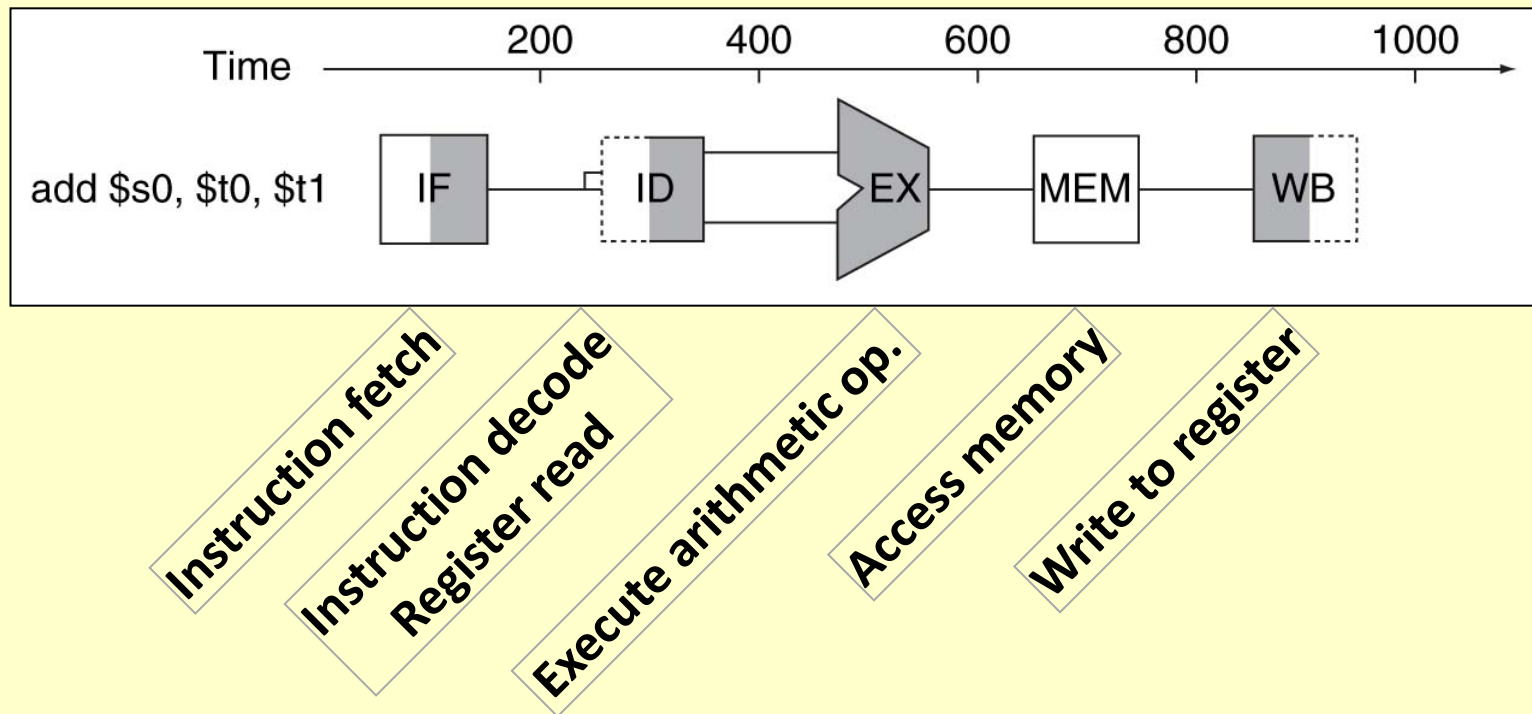
■ Write-back to Register

- Store result into register again

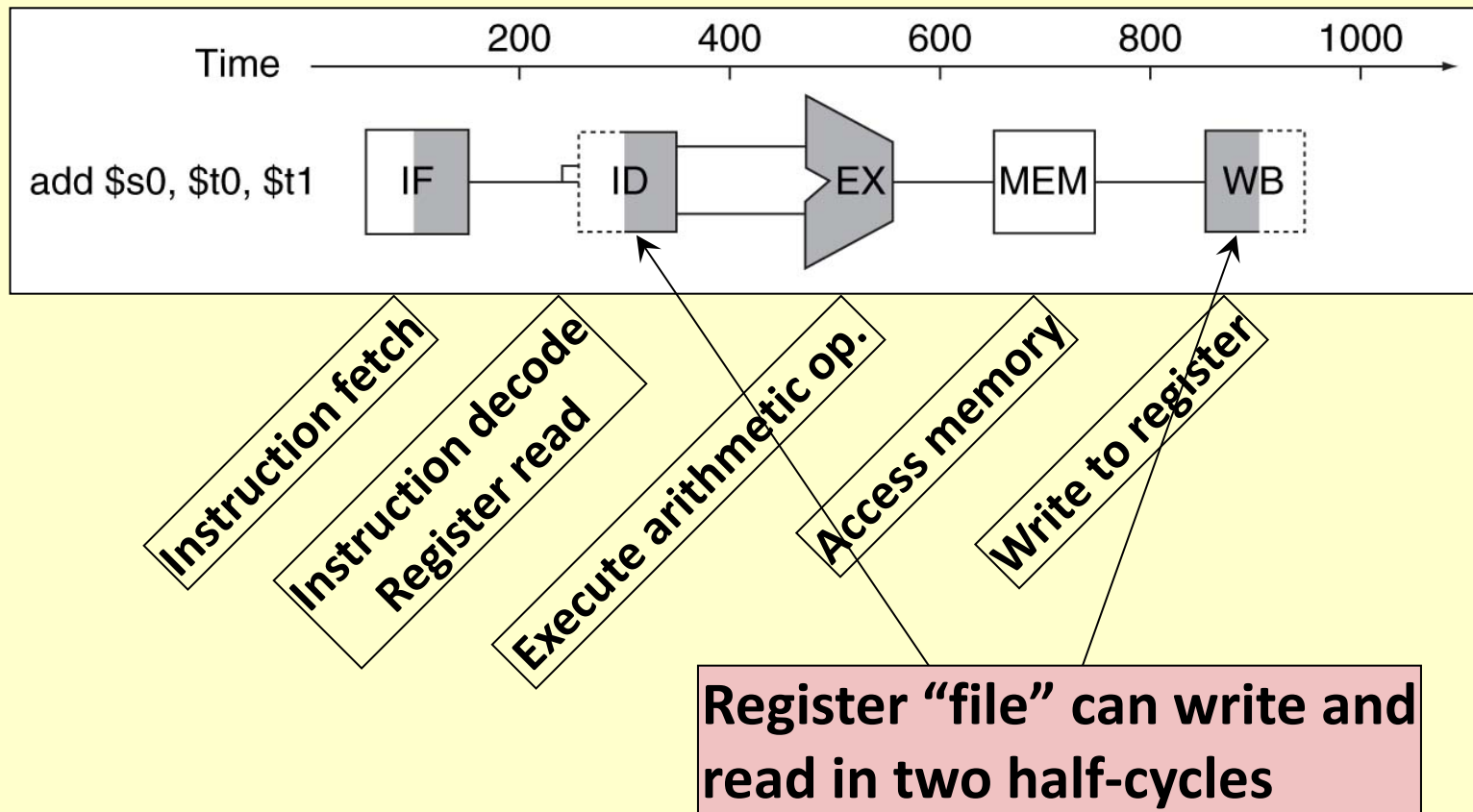
Every architecture partitions its instruction set differently

These are the five cycles of a many “textbook” pipelines!

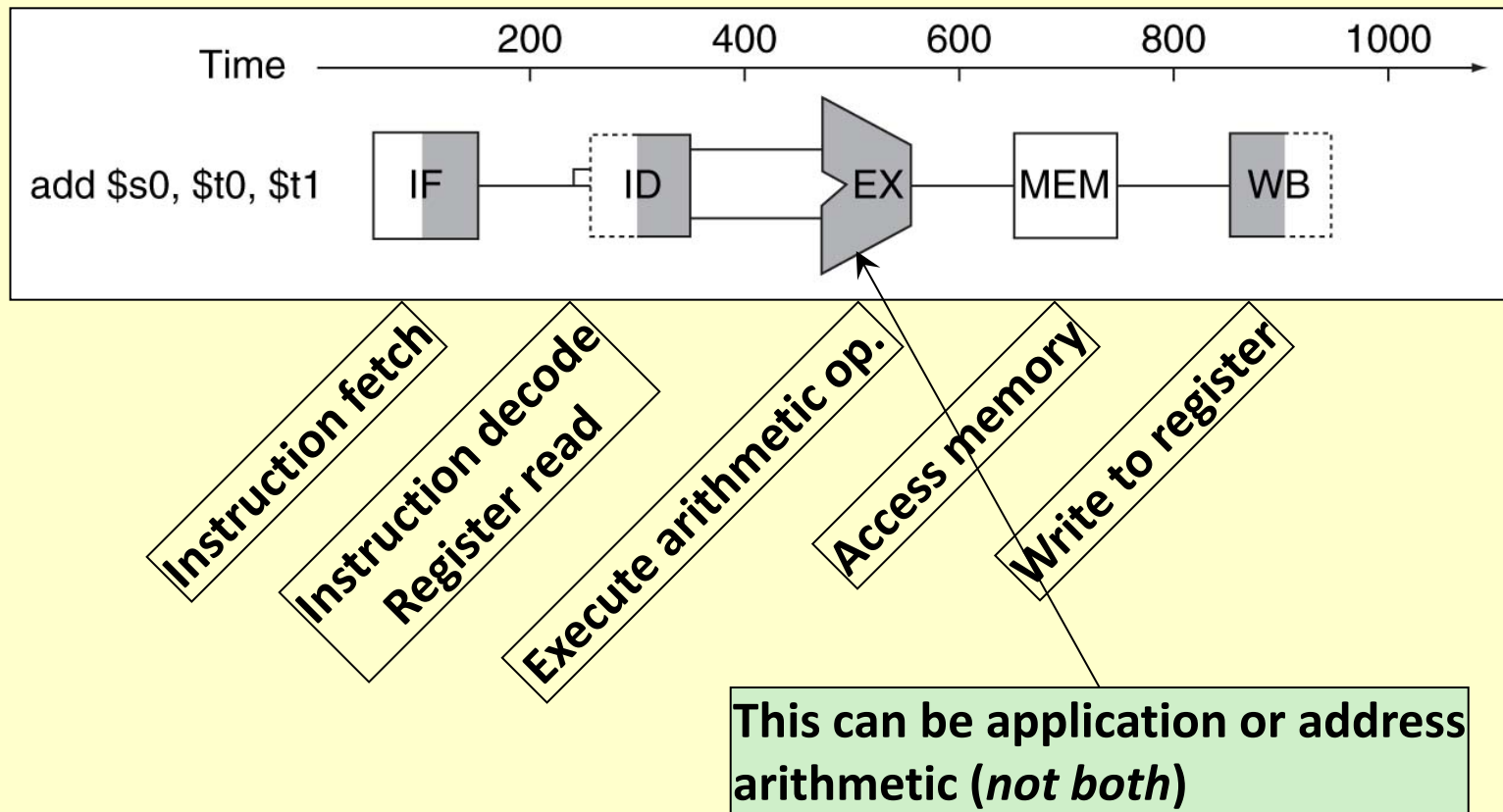
Example — MIPS Pipeline (continued)



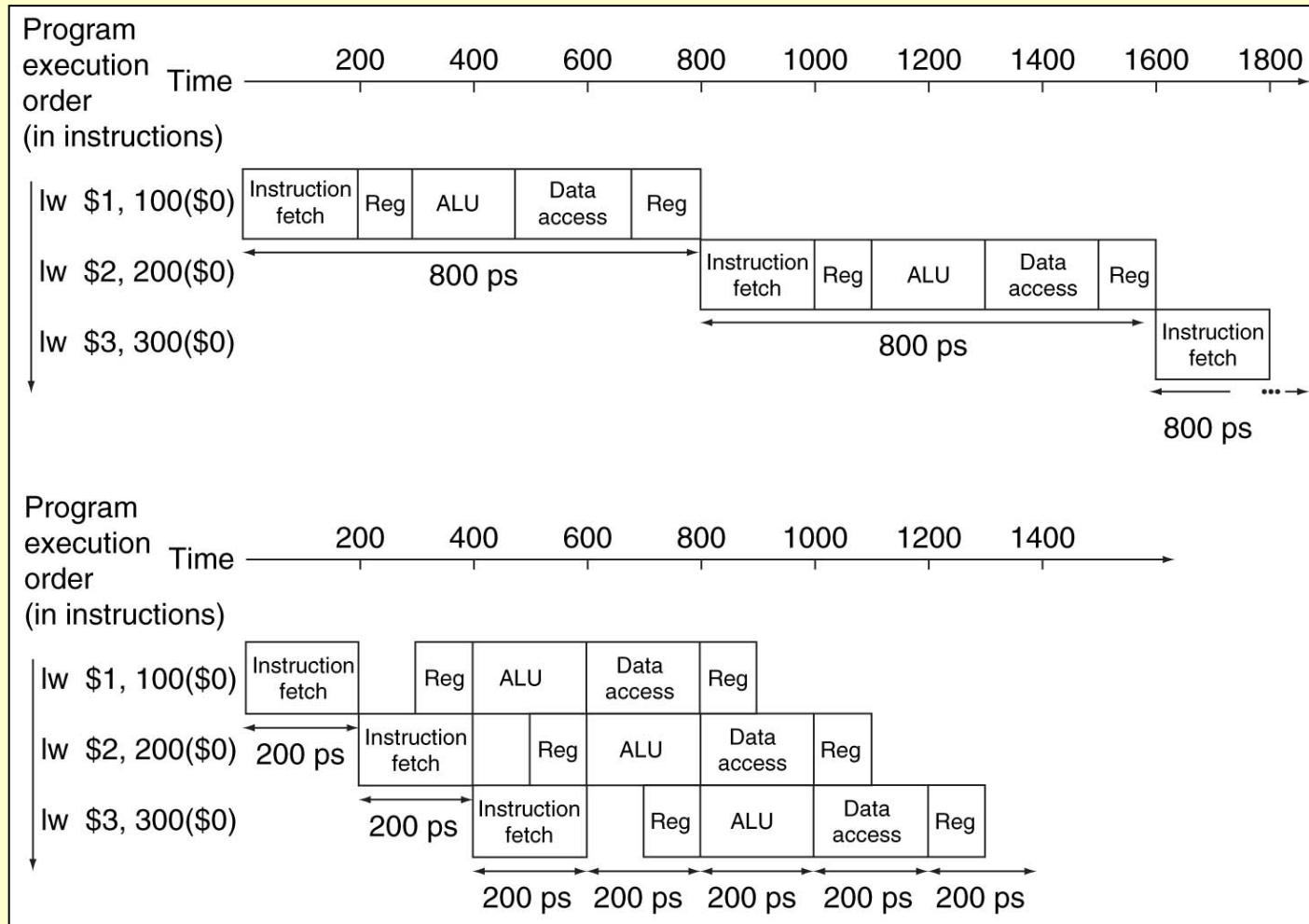
Example — MIPS Pipeline (continued)



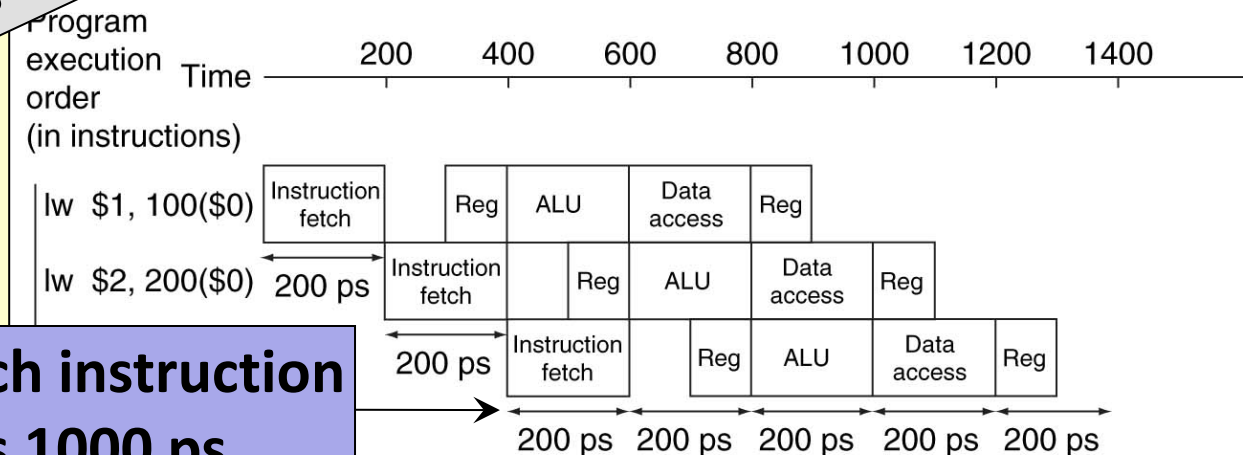
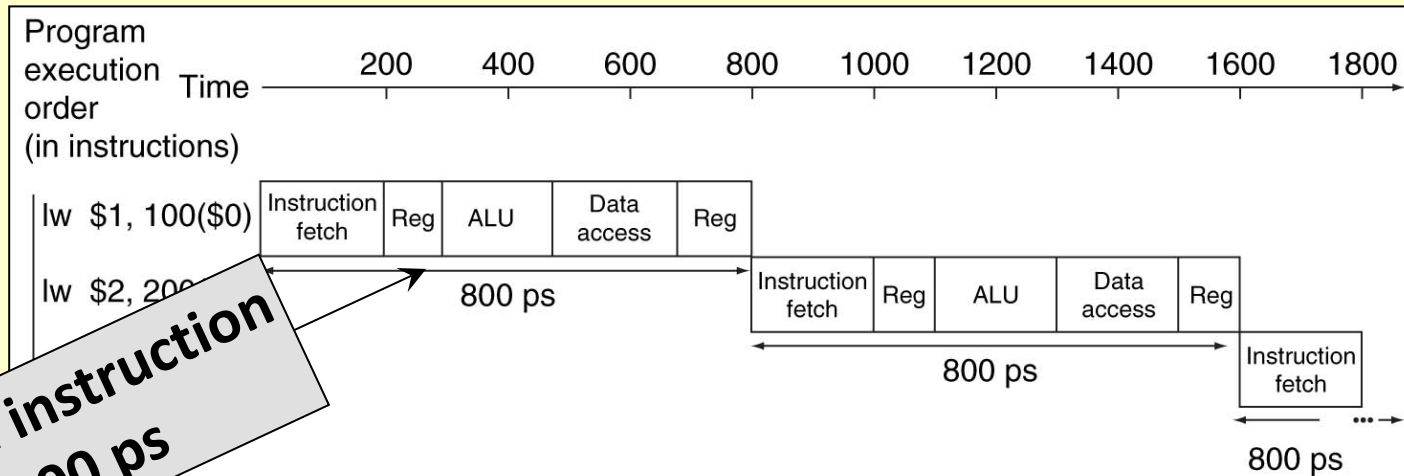
Example — MIPS Pipeline (continued)



Example — MIPS Pipeline (continued)

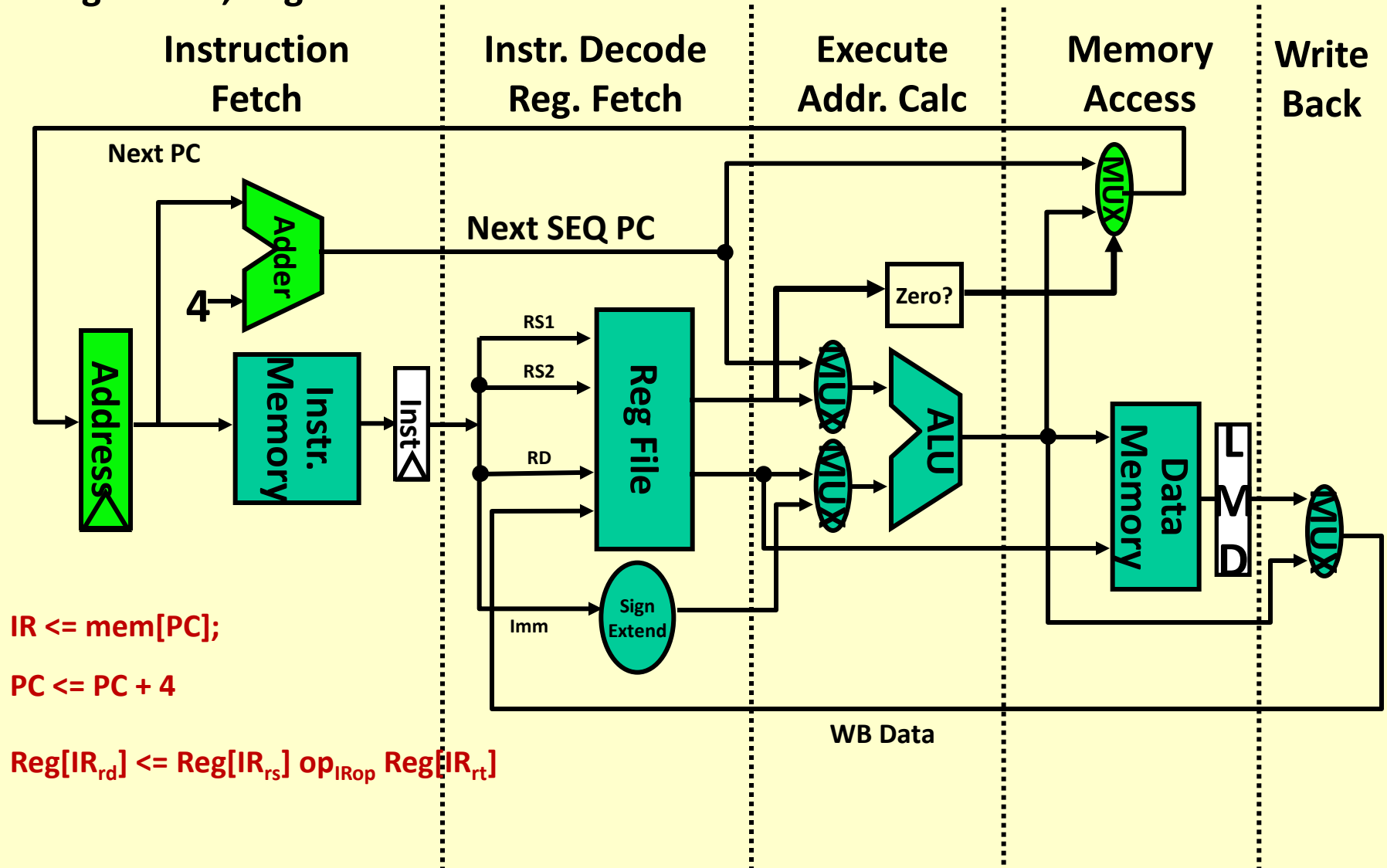


Example — MIPS Pipeline (continued)



MIPS Datapath (in five steps)

Figure C.2, Page A-8



MIPS Datapath (showing pipeline registers)

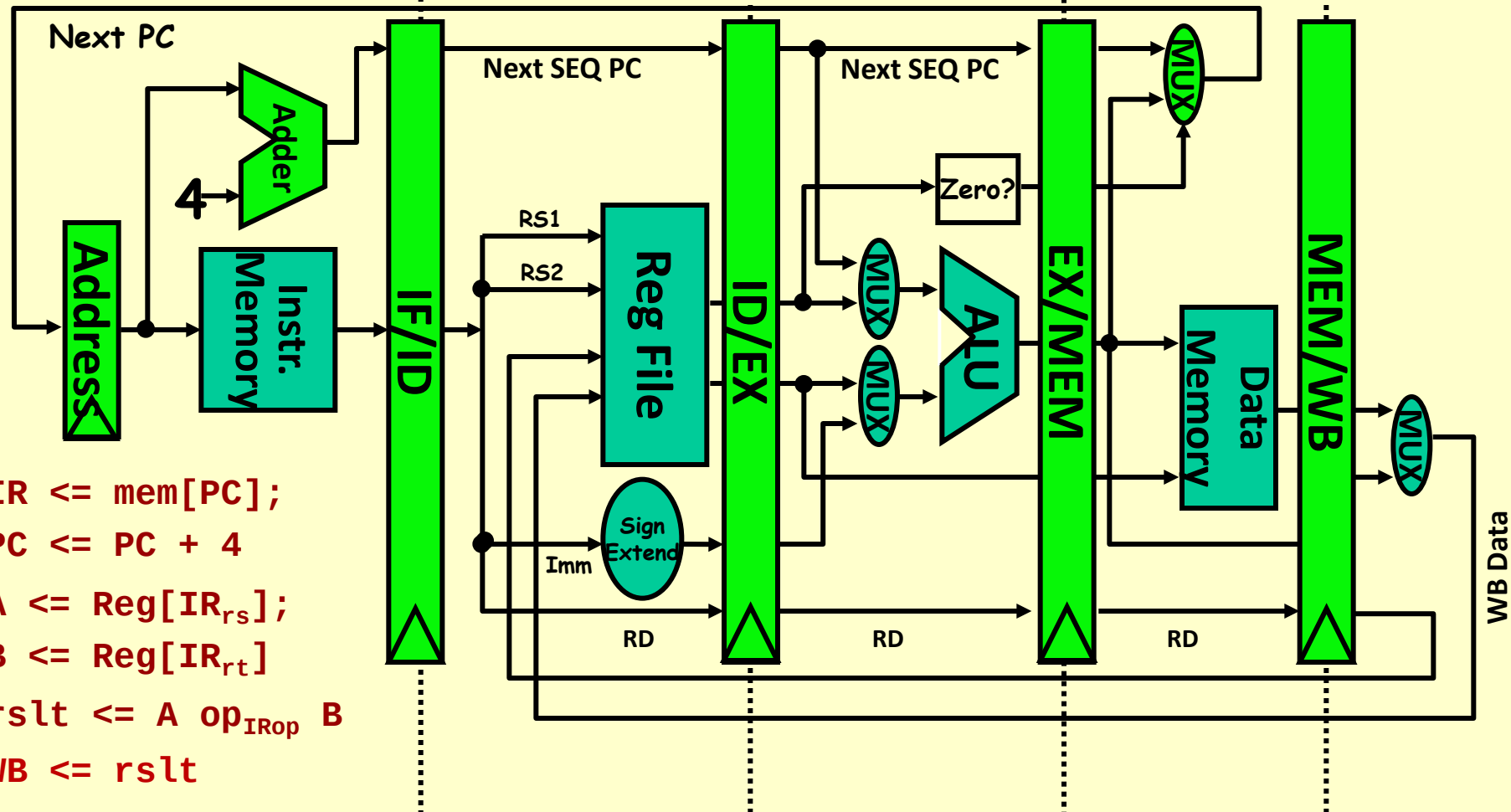
Figure C.3, Page C-9
Instruction
Fetch

Instr. Decode
Reg. Fetch

Execute
Addr. Calc

Memory
Access

Write
Back

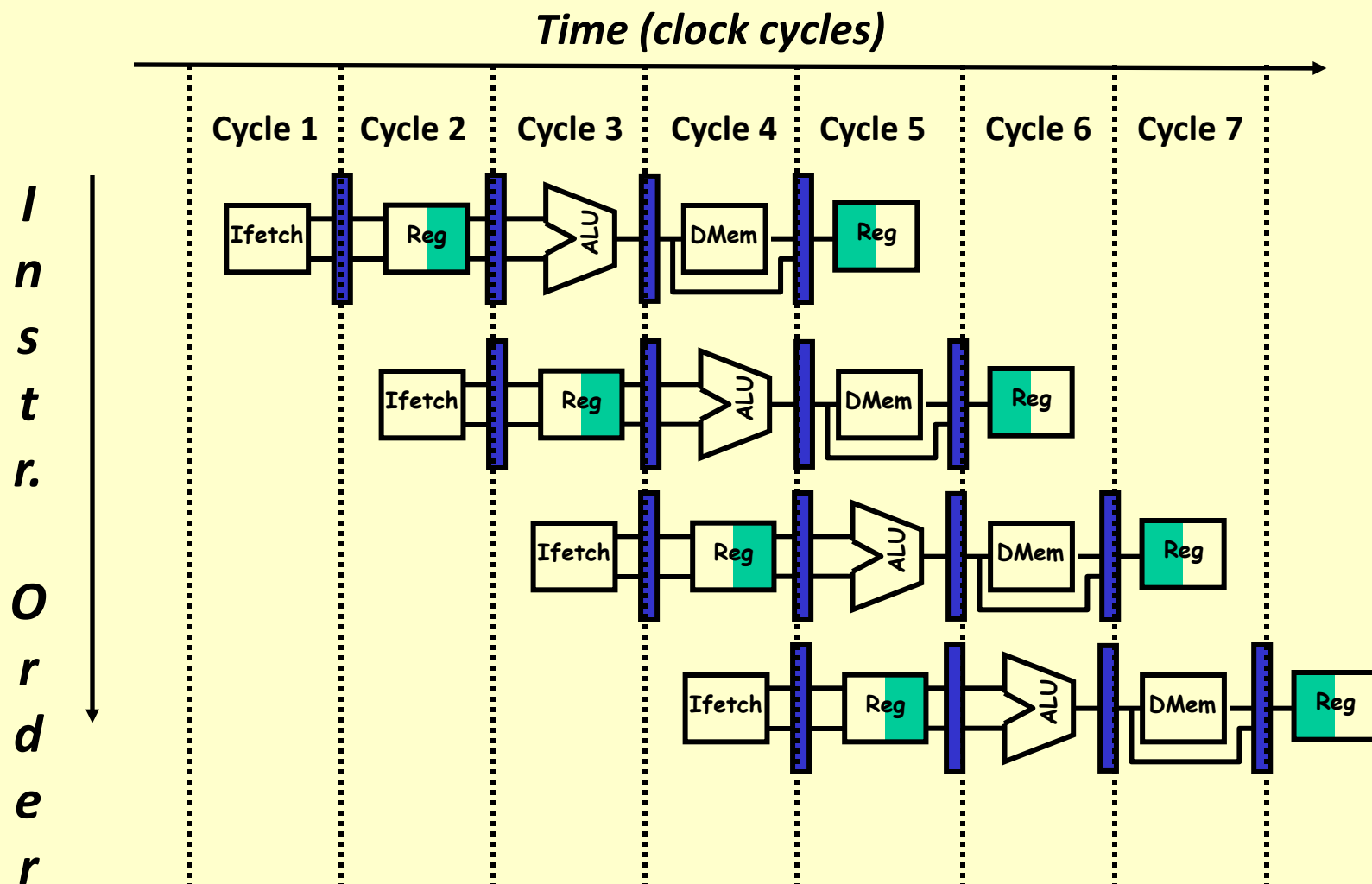


```

IR <= mem[PC];
PC <= PC + 4
A <= Reg[IRrs];
B <= Reg[IRrt];
rslt <= A opIRop B
WB <= rslt
Reg[IRrd] <= WB
  
```

Visualizing Pipelining

Figure c.2, Page c-8



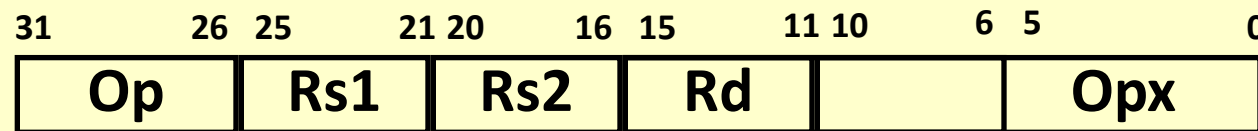
More on MIPS Pipeline

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

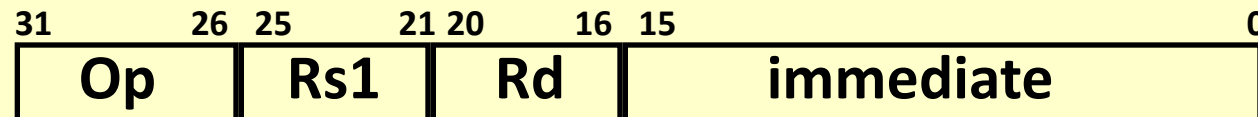
Digression – MIPS Instruction Formats

Fig A.22

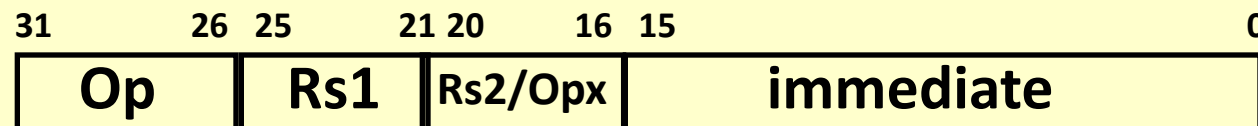
Register-Register



Register-Immediate



Branch



Jump / Call



More on MIPS Pipeline

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Quantifying the Speedup

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

■ Assume

- 40% ALU operations
- 20% branches
- 30% Load operations
- 10% Store operations
- No pipeline penalty

■ Average (non-pipelined) instruction duration = 650 ps

$$.4 \times 600 + .2 \times 500 + .3 \times 800 + .1 \times 700 = 650$$

Quantifying the Speedup (continued)

$$\text{Speedup} = \frac{\text{AveUnPipelinedTime}}{\text{AvePipelinedTime}} = \frac{650}{200} = 3.25$$

- **Unpipelined architecture would allow variable duration instructions**
 - Branches much faster than Loads
- **Pipelined architecture requires every cycle to take exactly the same time**
 - Some wasted time in Branch, ALU, and Store operations
- **Speedup is less if there is a penalty for pipelining**

More on Speedup

- Suppose that there is a penalty for including pipelining (vs. non-pipelined design)
- The clock cycle of each stage in pipelined architecture would be slightly longer
- Speedup would be less
- Subject to Amdahl's Law

- See pp. C-10 & C.11

Questions?

Life is Never that Simple!

Definition!

- Introducing *hazards*
- I.e., factors that interfere with full and efficient pipelining
- Prevent instruction from starting or continuing on next cycle

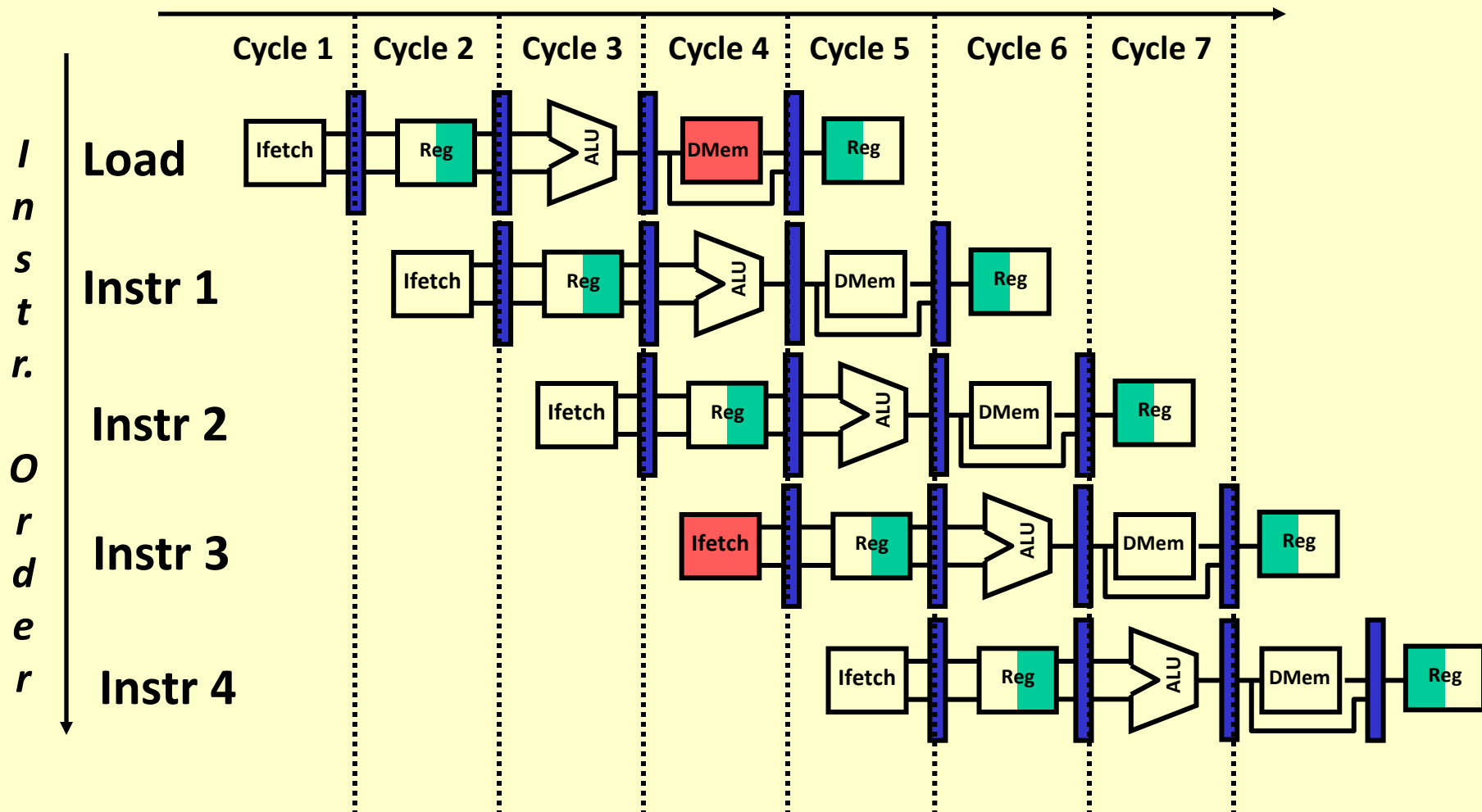
Hazards

- ***Structural*** – resource conflicts among successive instructions
 - Hardware cannot support all possible combinations of instructions in rapid succession
- ***Data*** – dependencies of instructions on results of other instructions
 - $x = a * b + c$ Cannot add until multiply is done
- ***Control*** – branches that change instruction fetch order
 - Will affect instructions that have already been fetched!

Structural Hazard Example:–

One Memory Port Figure C.4, Page C-14

Time (clock cycles)

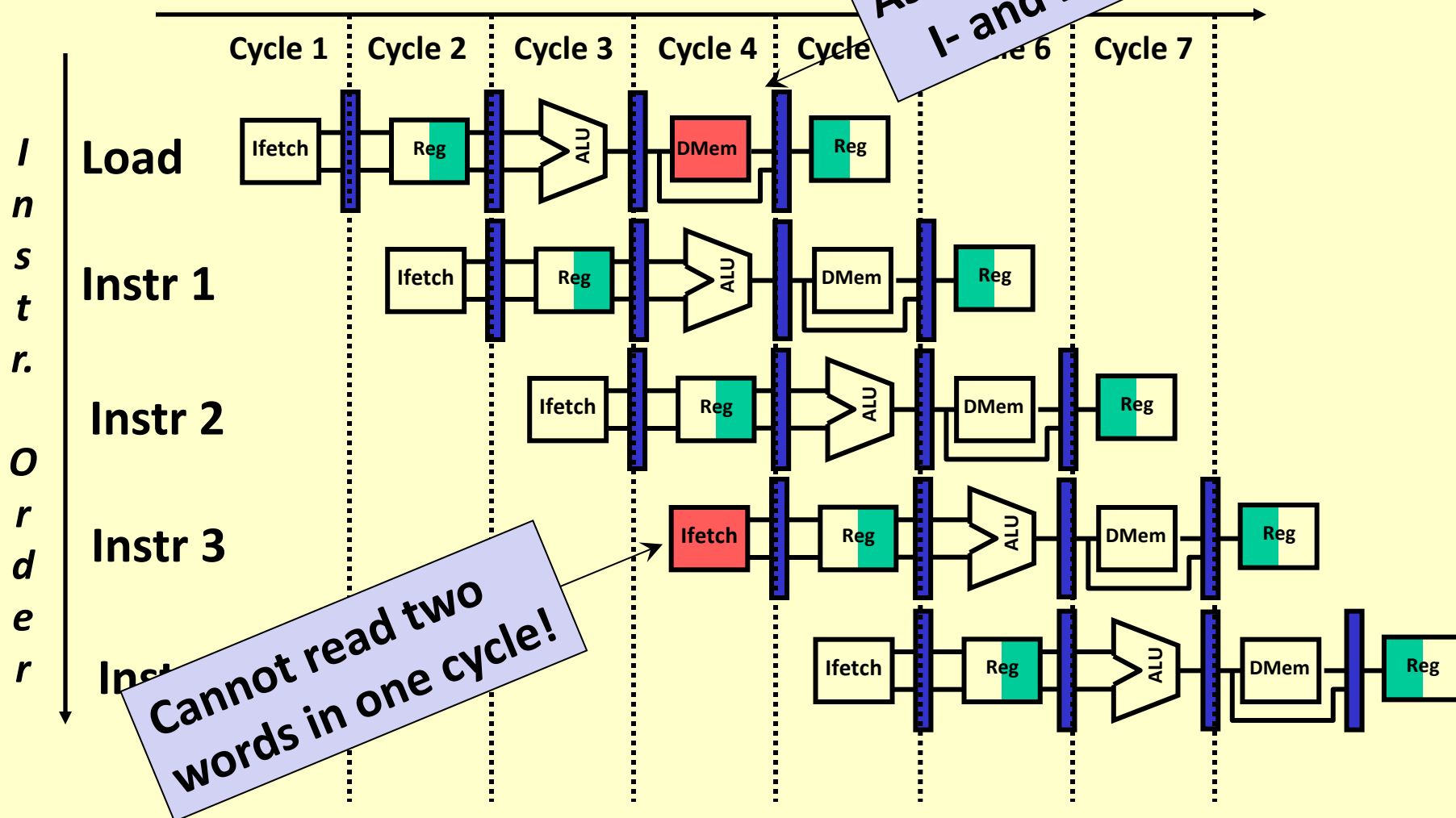


Structural Hazard Example:–

One Memory Port

Figure C.4, Page C-14

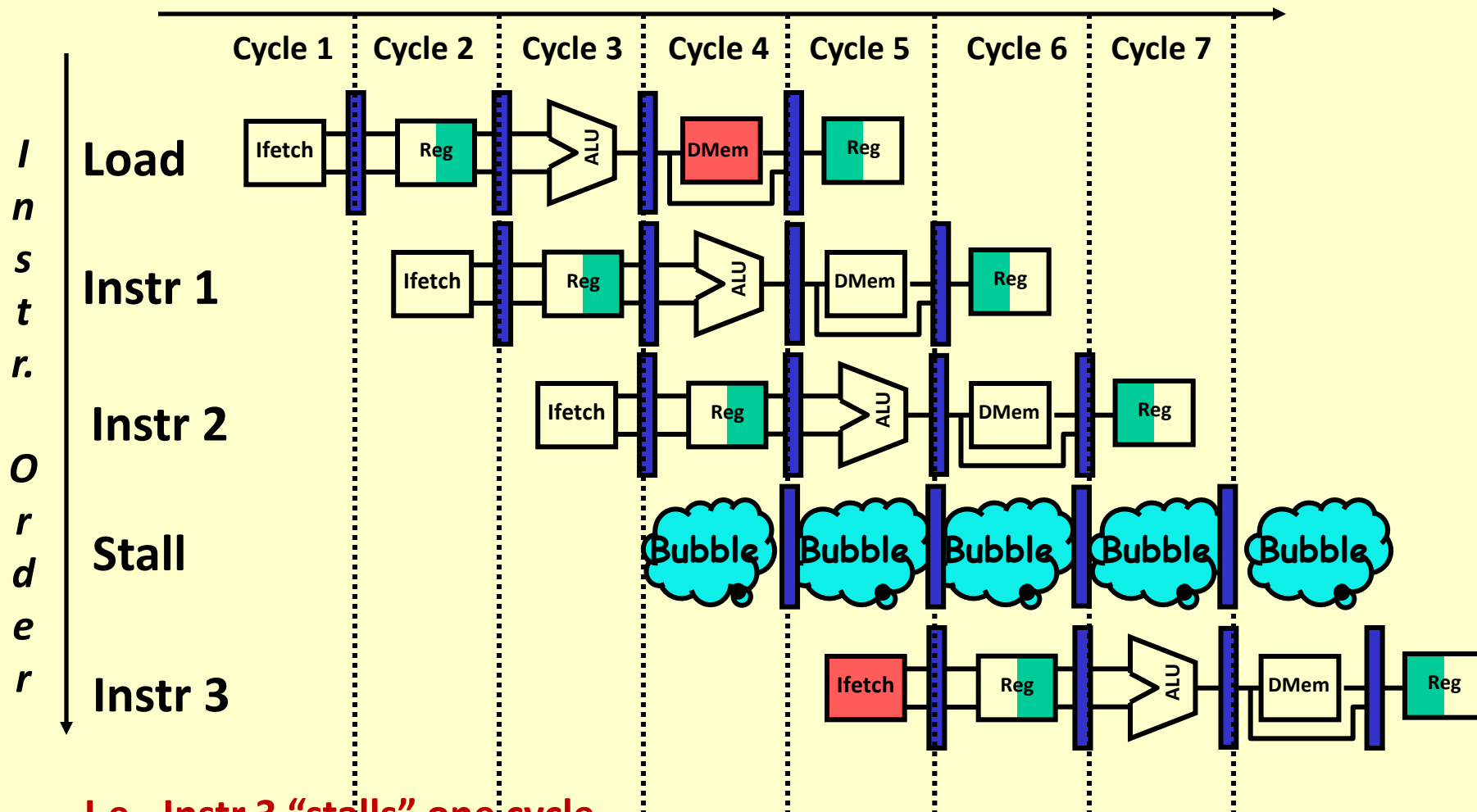
Time (clock cycles)



Penalty – a Bubble in Pipeline

(Similar to Figure C.5, Page C-15)

Time (clock cycles)



Speedup Equation for Pipelining

See pp. C-12 & C-13

$$CPI_{\text{pipelined}} = \text{Ideal CPI} + \text{Average Stall cycles per Inst}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

For simple RISC pipeline, CPI = 1

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Cycle Time}_{\text{unpipelined}}}{\text{Cycle Time}_{\text{pipelined}}}$$

Quantifying Dual-port vs. Single-port

- **Machine A: Dual ported memory**

- aka “Harvard Architecture”

- **Machine B: Single ported memory**

- but pipelined implementation has $1.05 \times$ faster clock rate

- **Ideal CPI = 1 for both**

- **Loads are 40% of instructions executed**

$$\begin{aligned} \text{Speedup}_A &= \text{Pipeline Depth} / (1 + 0) \times (\text{clock}_{\text{unpipe}} / \text{clock}_{\text{pipe}}) \\ &= \text{Pipeline Depth} \end{aligned}$$

$$\begin{aligned} \text{Speedup}_B &= \text{Pipeline Depth} / (1 + 0.4 \times 1) \times (\text{clock}_{\text{unpipe}} / (\text{clock}_{\text{unpipe}} / 1.05)) \\ &= (\text{Pipeline Depth} / 1.4) \times 1.05 \\ &= 0.75 \times \text{Pipeline Depth} \end{aligned}$$

$$\text{Speedup}_A / \text{Speedup}_B = \text{Pipeline Depth} / (0.75 \times \text{Pipeline Depth}) = 1.33$$

- **Machine A is 1.33 times faster**

What to do about Memory Structural Hazard

- **Typical processor has two memory ports ...**

- I-port
- D-port

- **... or two separate L1 cache memories**

- I-cache
- D-cache

Other Structural Hazards (examples)

■ Multiply

- Expensive in gates to make fully pipelined

■ Floating-point divide

- Very expensive to make fully pipelined

■ Floating-point square-root

- Prohibitively expensive to pipeline at all

Structural Hazards (conclusion)

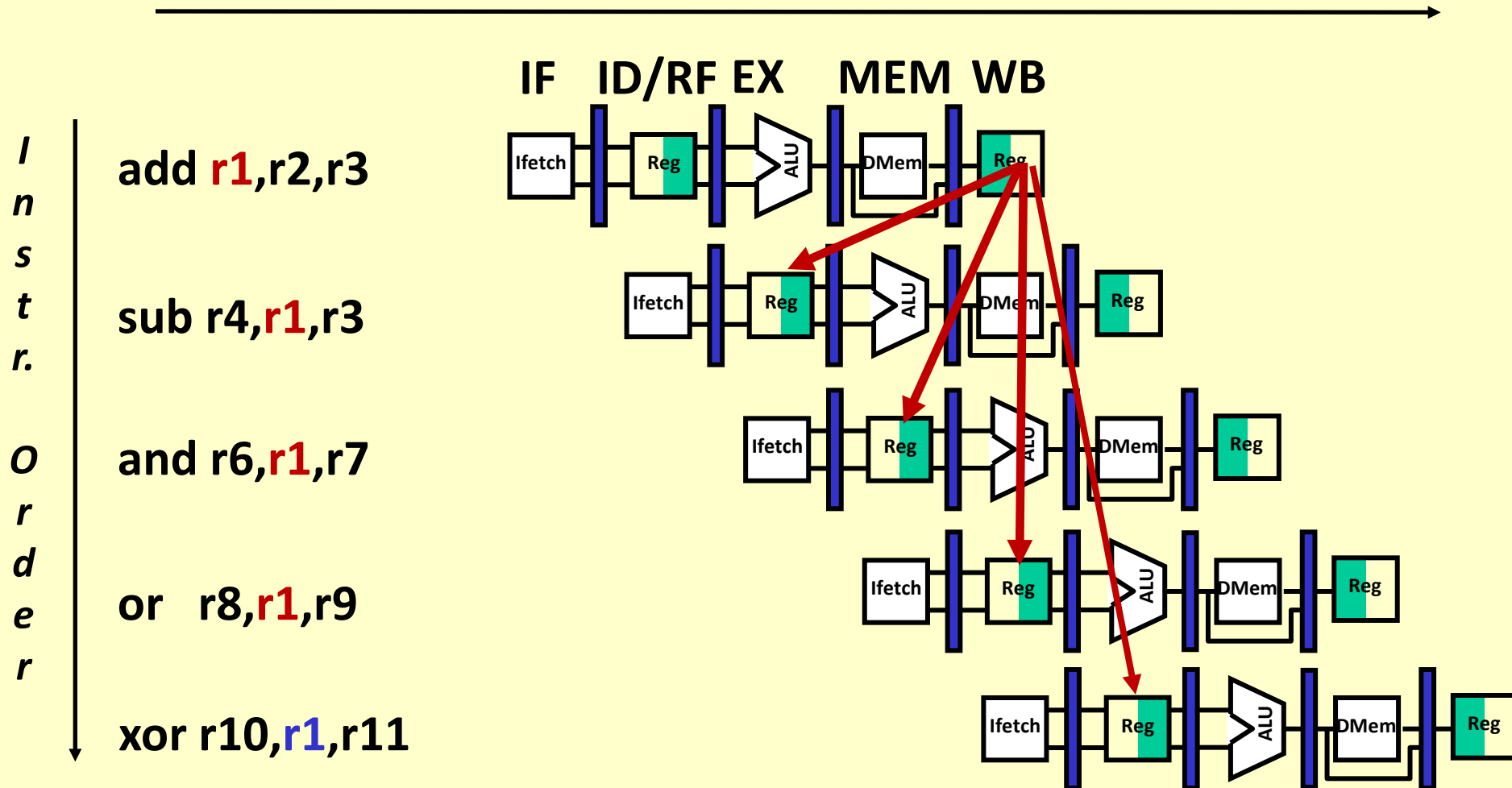
- **Processor hardware *must* check**
 - Introduce “stalls”
- **Compiler *should be* aware**
 - Re-arrange compiled code

Questions?

Data Hazard on R1

Figure C.6, Page C-17

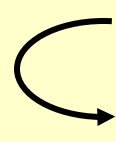
Time (clock cycles)



Three Generic Data Hazards (1)

■ Read After Write (RAW)

Instr_i, tries to read operand before *Instr_j*, writes it

 I: add **r1**, r2, r3
J: sub r4, **r1**, r3

■ Caused by a “**Dependence**” (in compiler nomenclature)

- This hazard results from an actual need for data communication among instructions

Three Generic Data Hazards (2)

■ Write After Read (WAR)

Instr_j writes new value before Instr_i reads old value

```
I: sub r4, r1, r3
J: add r1, r2, r3
K: mul r6, r1, r7
```

- Called an “**anti-dependence**” by compiler writers. This results from reuse of the name “**r1**”.

■ Can't happen in MIPS 5 stage pipeline because:–

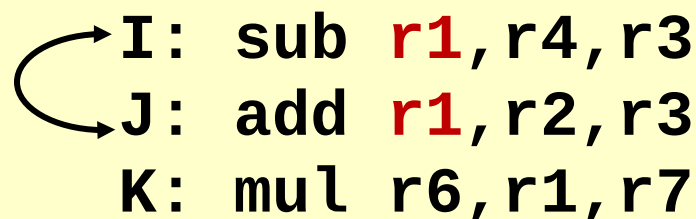
- All instructions take 5 stages, and
- Reads are always in stage 2, and
- Writes are always in stage 5

- Many other processors susceptible to this hazard

Three Generic Data Hazards (3)

- **Write After Write (WAW)**

Instr_j writes a value before *Instr_i* writes a different value



```
I:  sub  r1, r4, r3
J:  add  r1, r2, r3
K:  mul  r6, r1, r7
```

- Called an “**output dependence**” by compiler writers

This also results from the reuse of name “**r1**”.

- Can’t happen in MIPS 5 stage pipeline because:

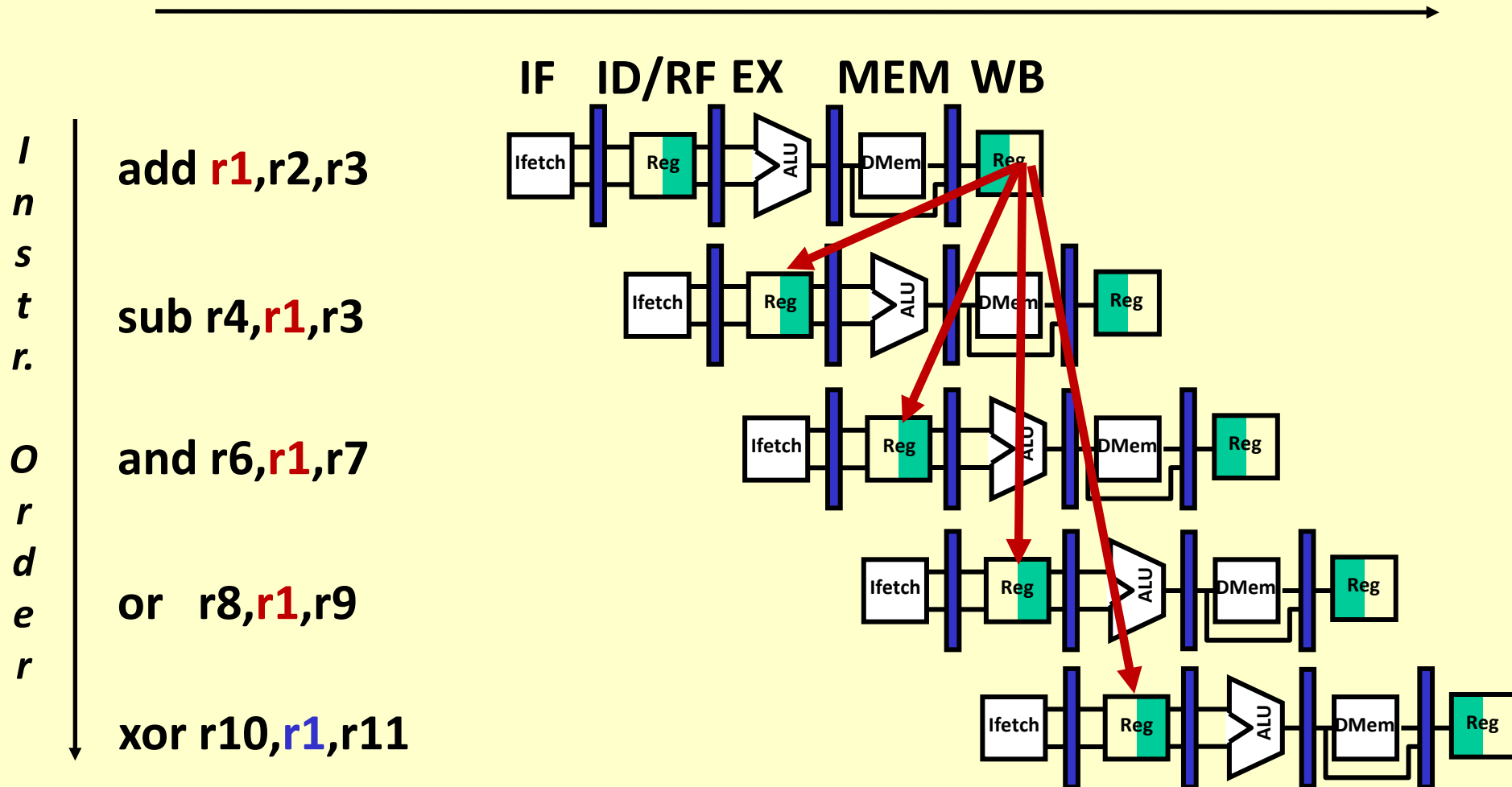
- All instructions take 5 stages, and
- Writes are always in stage 5

- Will see WAR and WAW in more complicated pipelines

Data Hazard on R1

Figure C.6, Page C-17

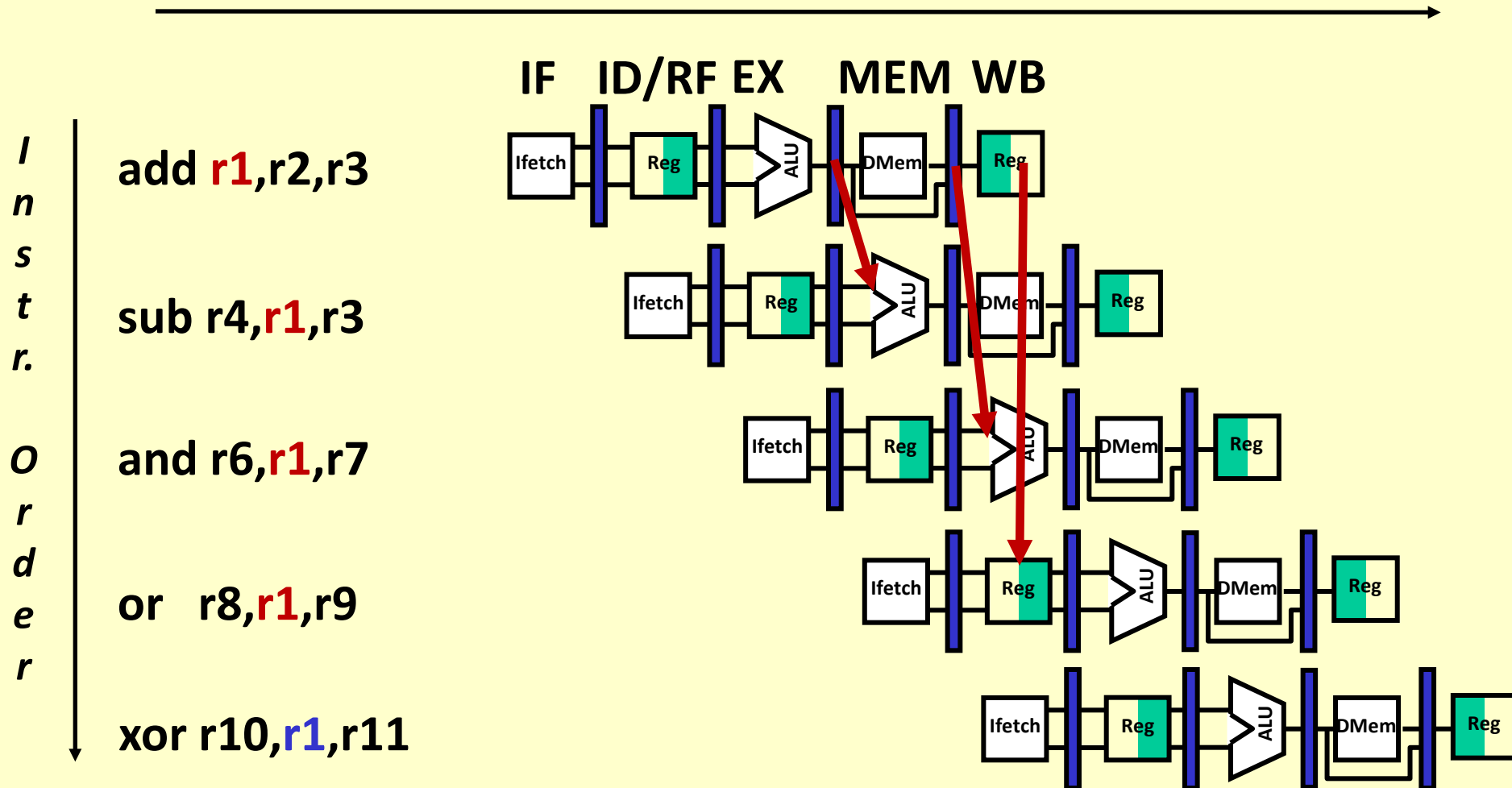
Time (clock cycles)



Forwarding to Avoid Data Hazard

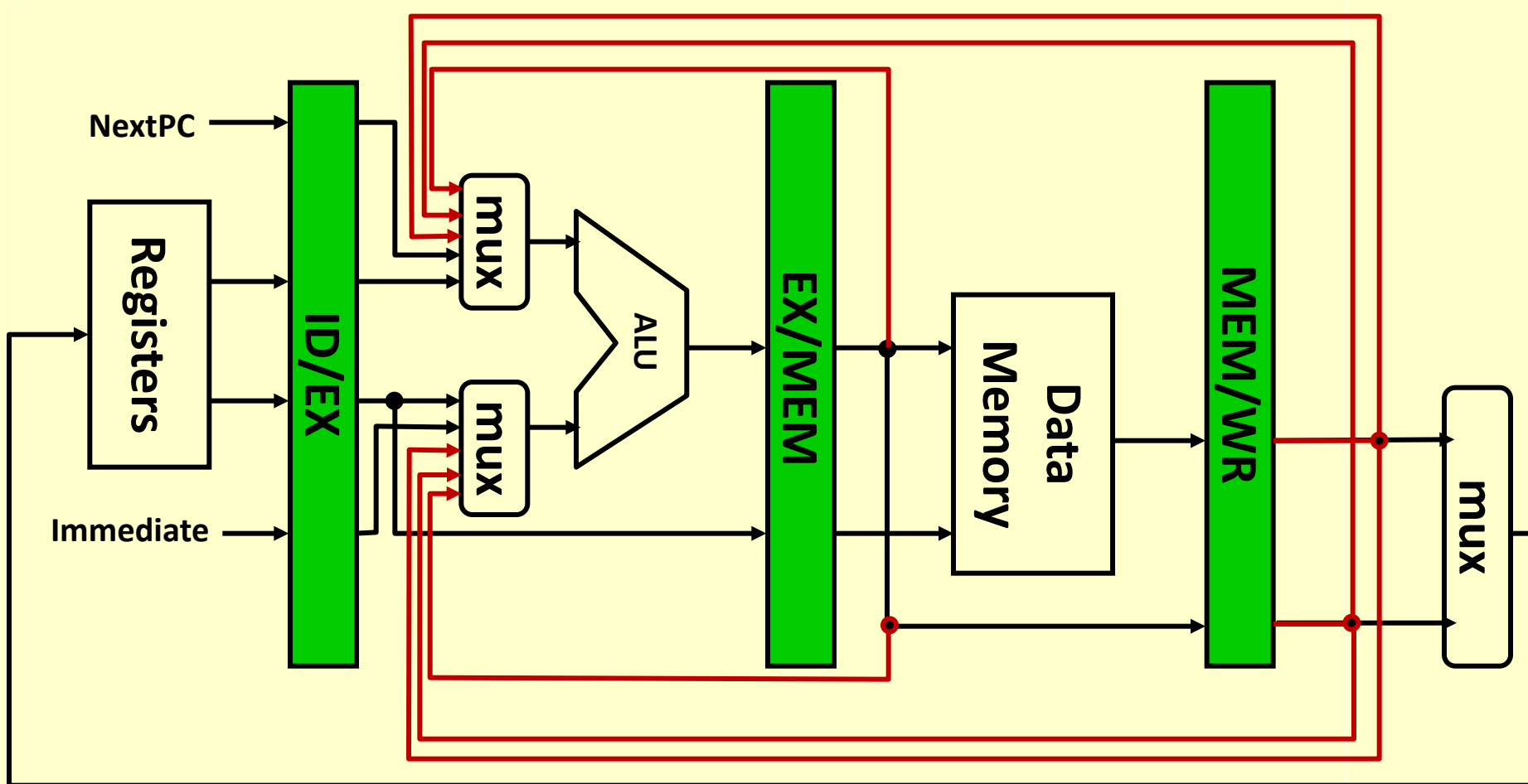
(Figure C.7, Page C-18)

Time (clock cycles)



HW Change for Forwarding

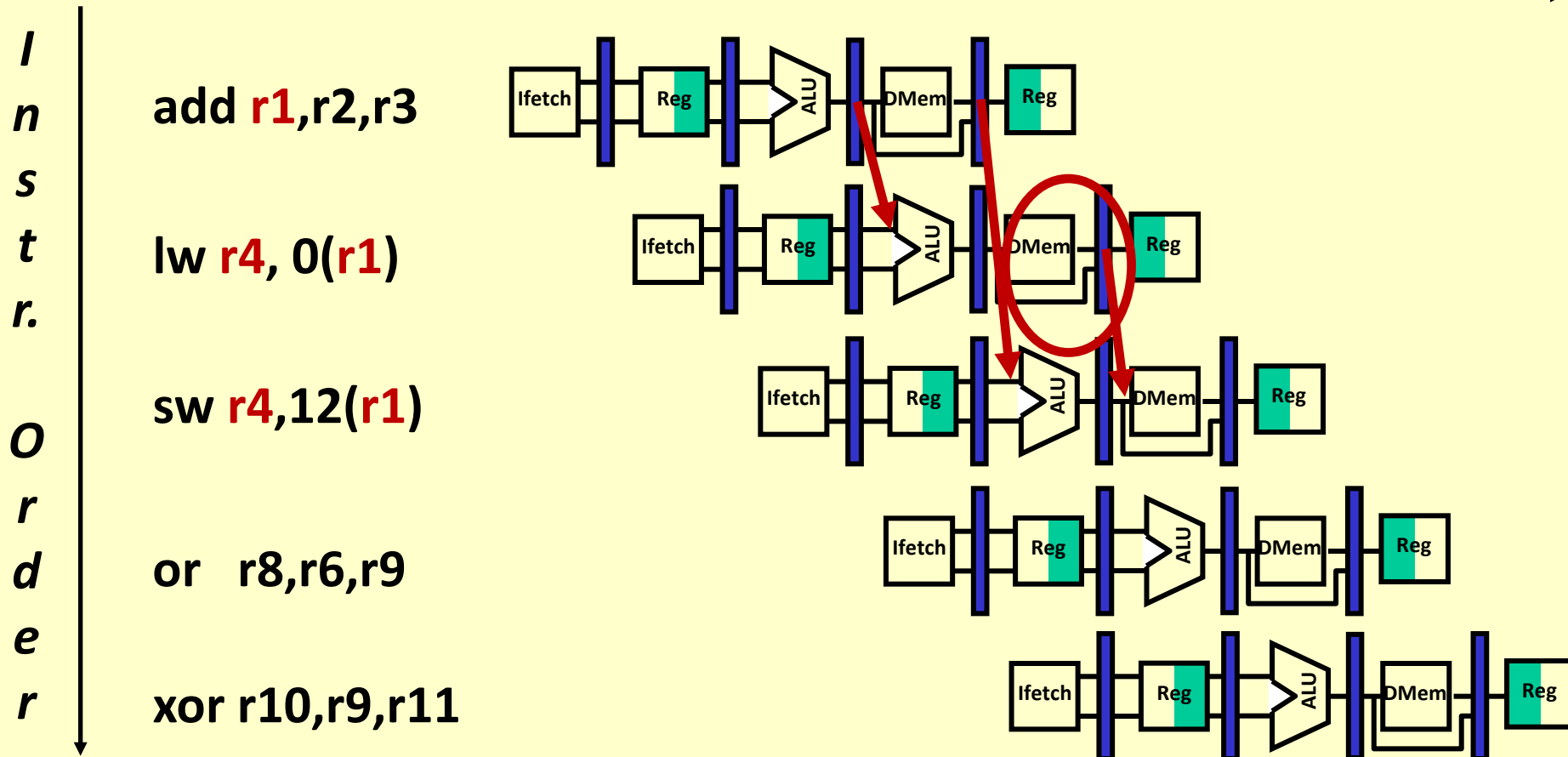
Figure C.21, Page C-34



Forwarding to Avoid LW-SW Data Hazard

Figure C.8, Page C-19

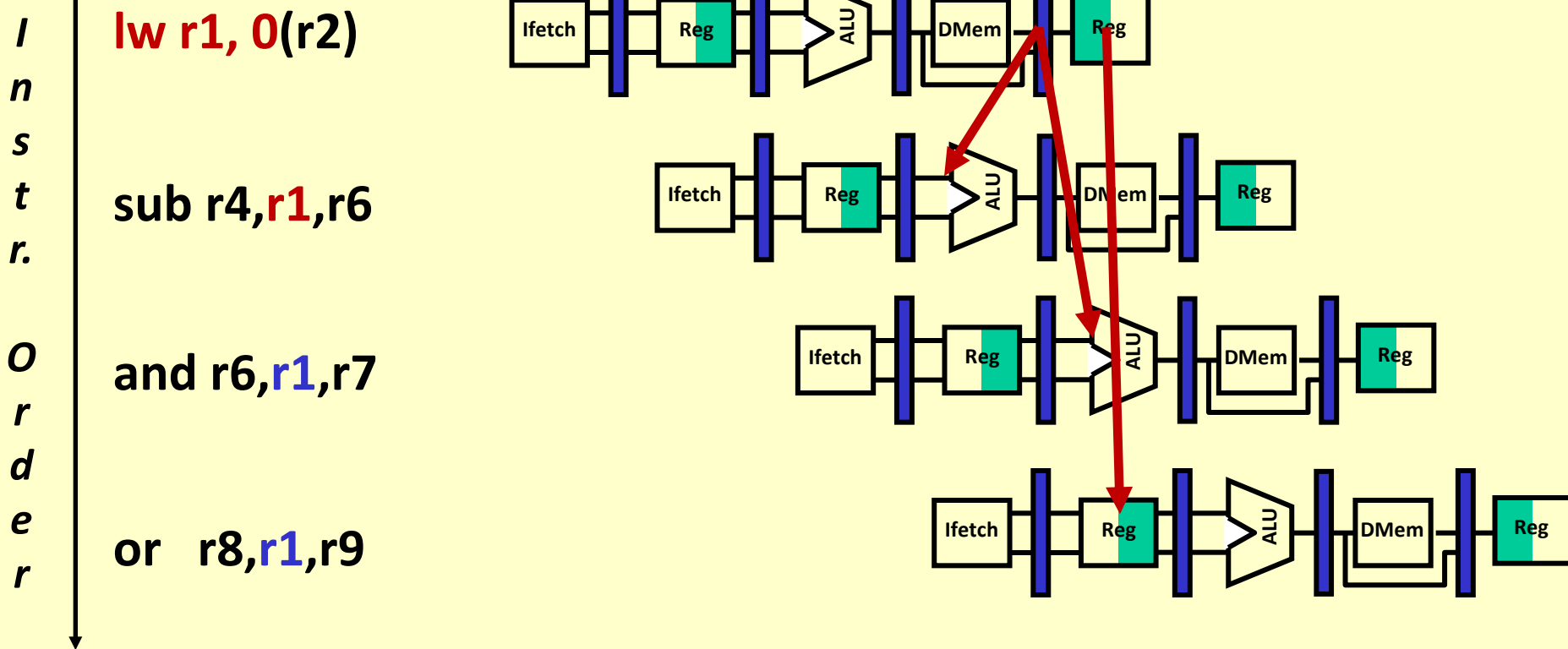
Time (clock cycles) →



Data Hazard Even with Forwarding

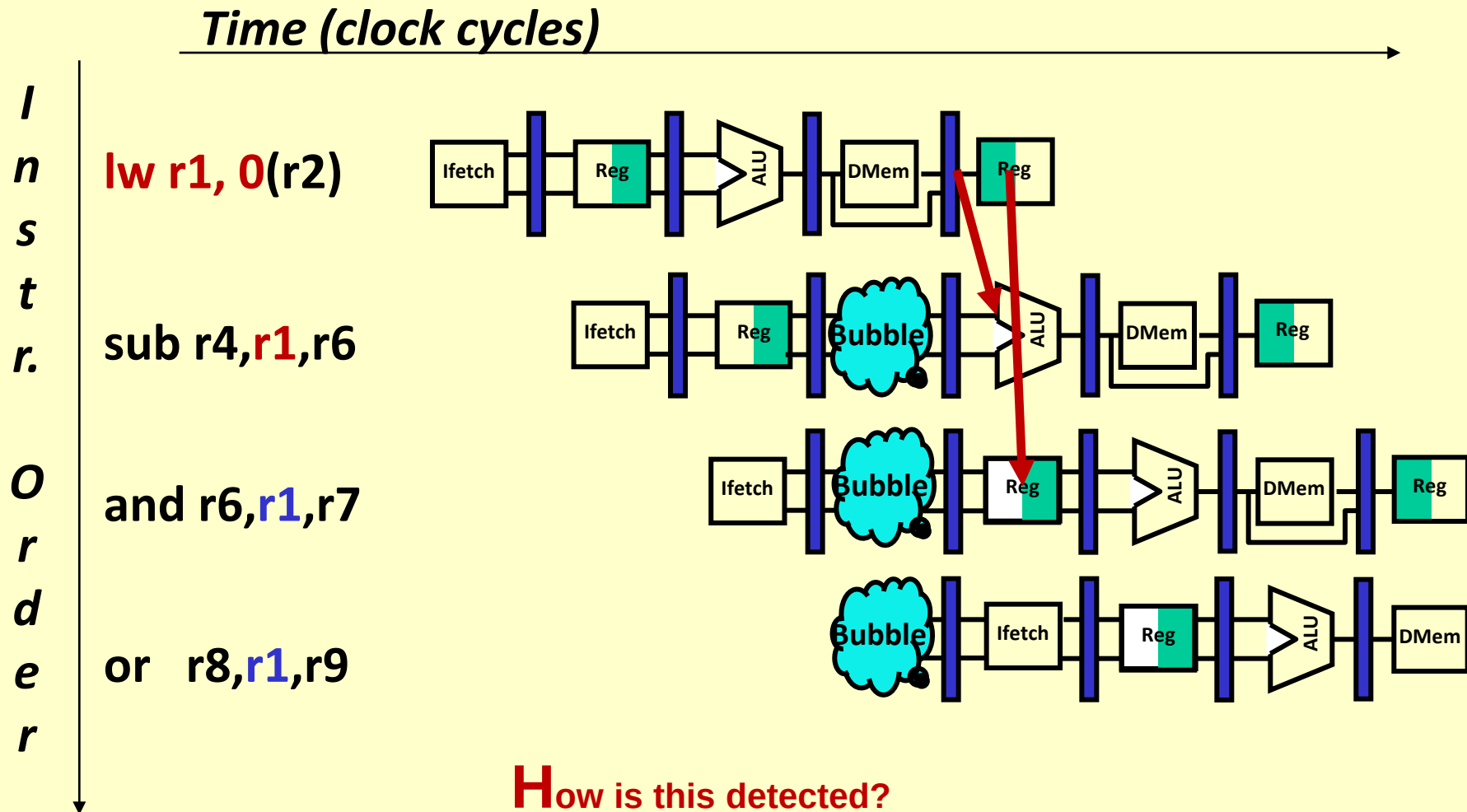
(Figure C.9, Page C-20)

Time (clock cycles)



Data Hazard Causing Pipeline Stall

(See Figure C.10, Page C-21)



See below

Software Scheduling to Avoid Data Hazards

Try producing fast code for

$a = b + c;$

$d = e - f;$

assuming $a, b, c, d, e,$ and f in memory.

Slow code:

LW Rb,b

LW Rc,c

ADD Ra,Rb,Rc

SW a,Ra

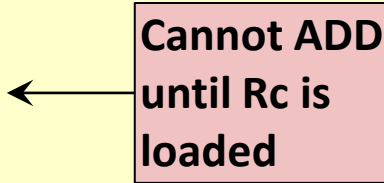
LW Re,e

LW Rf,f

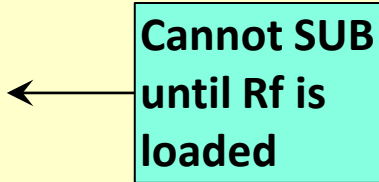
SUB Rd,Re,Rf

SW d,Rd

Cannot ADD
until Rc is
loaded

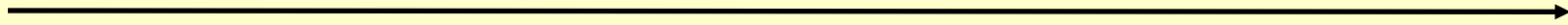


Cannot SUB
until Rf is
loaded



Data Hazard (continued)

Time (clock cycles)



*I
n
s
t
r.

O
r
d
e
r*

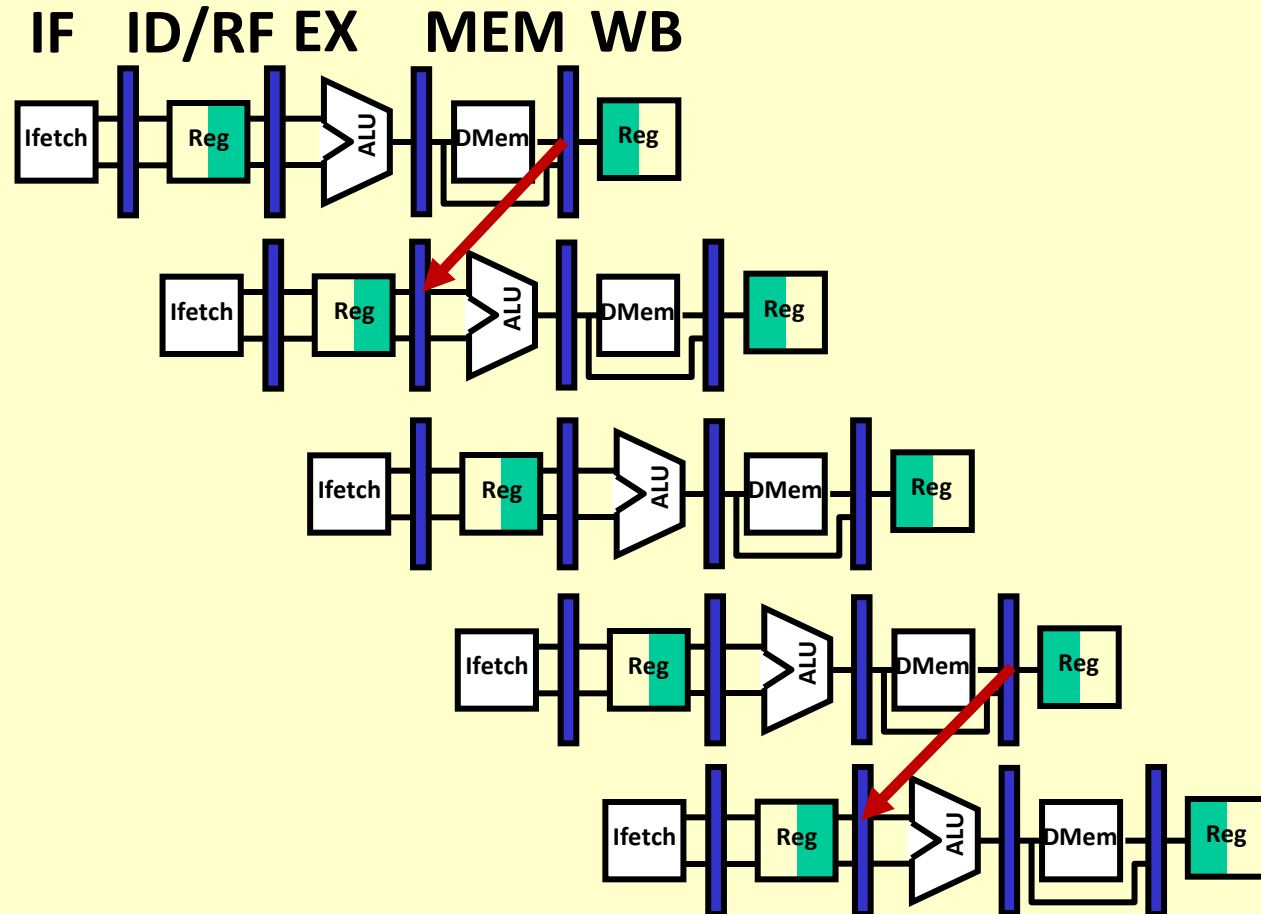
lw Rc, c

add Ra, Rb, Rc

...

lw Rf, f

sub Rd, Re, Rf



Software Scheduling to Avoid Data Hazards

Try producing fast code for

$a = b + c;$

$d = e - f;$

assuming $a, b, c, d, e,$ and f in memory.

Slow code:

```
LW  Rb,b
LW  Rc,c
ADD Ra,Rb,Rc
SW  a,Ra
LW  Re,e
LW  Rf,f
SUB Rd,Re,Rf
SW  d,Rd
```

Fast code:

```
LW  Rb,b
LW  Rc,c
LW  Re,e
ADD Ra,Rb,Rc
LW  Rf,f
SW  a,Ra
SUB Rd,Re,Rf
SW  d,Rd
```

A useful instruction

A useful instruction

Compiler optimizes for performance.
Hardware checks and interlocks for safety.

Questions?

Detecting and Interlocking for Stalls

- **Instruction Decode**

1. **Not used**

- **For each operand, determine where it comes from**

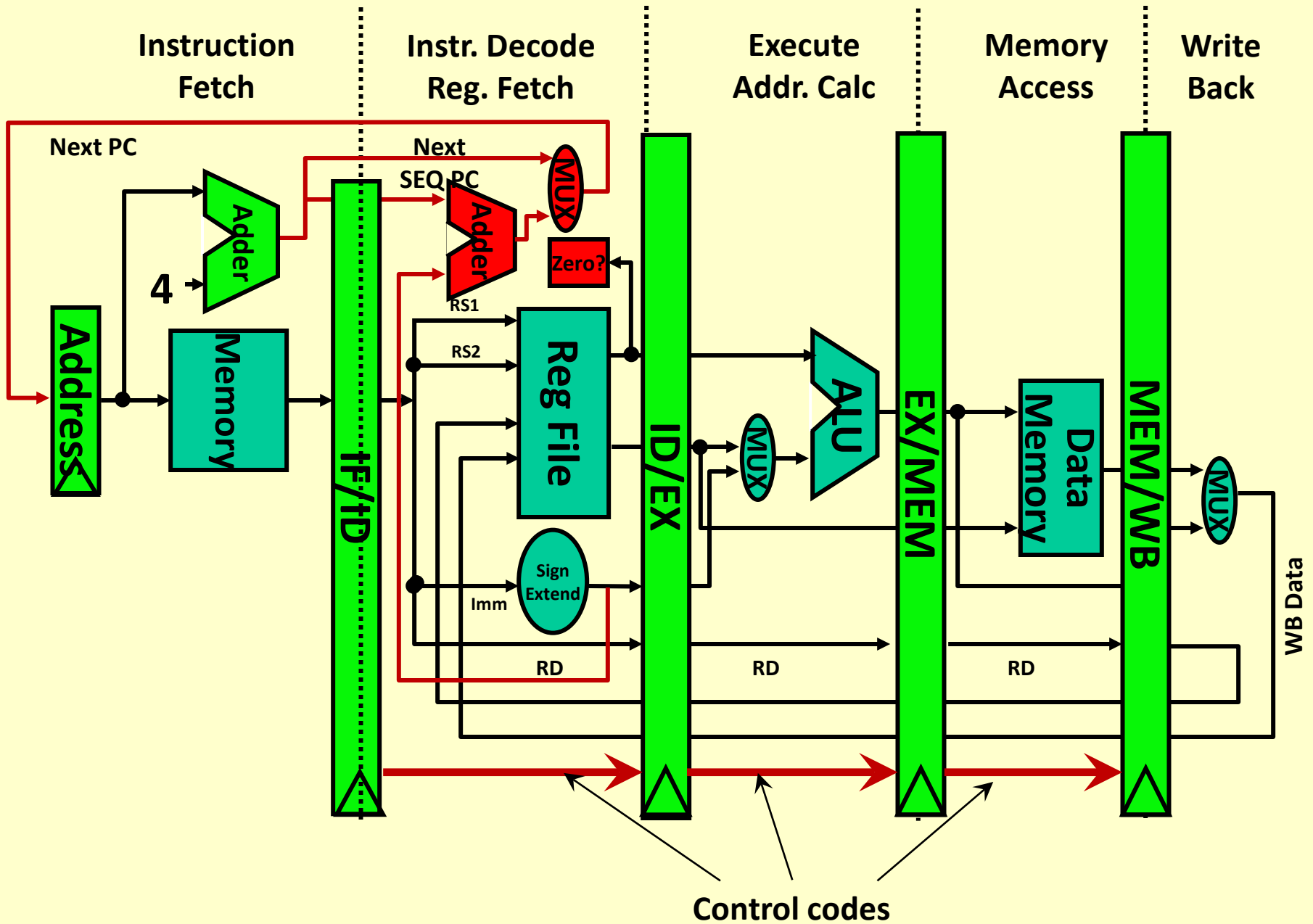
2. **Immediate**

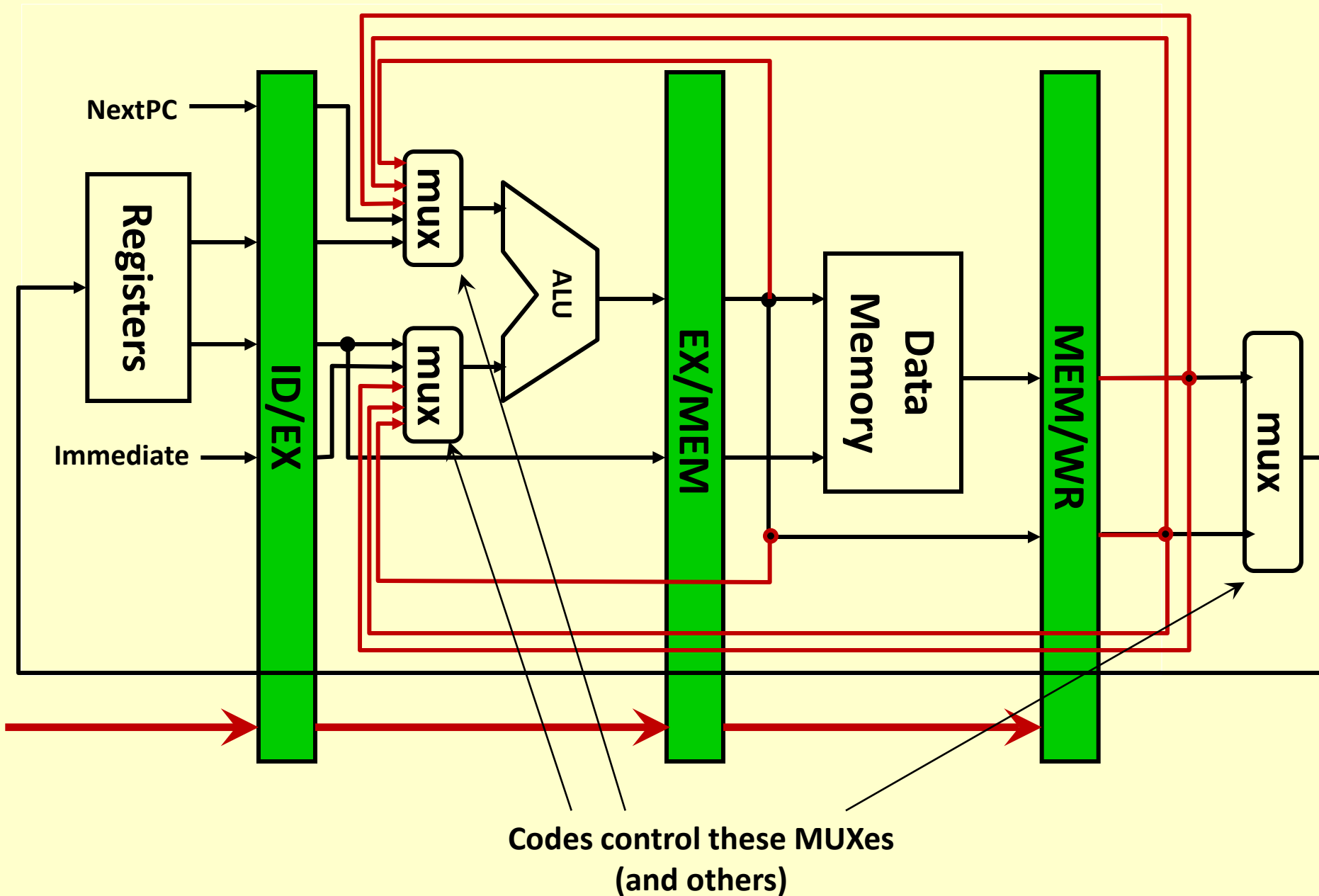
3. **Read from register**

- **Send control codes through pipeline**

4. **Forward from ALU stage**

5. **Forward from Mem stage**



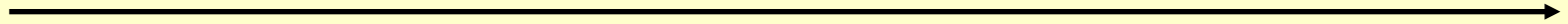


How to decide

- Keep track of three previous instructions
- If operand is *not* written by any of three previous instructions ...
 - ... then read from register
- If operand is output of ALU operation of *previous* instruction ...
 - ... then forward from ALU stage
- If operand is output from ALU or MEM of 2nd previous instruction ...
 - ... then forward from MEM stage
- Else stall one cycle

Interlocking for Data Hazards (continued)

Time (clock cycles)



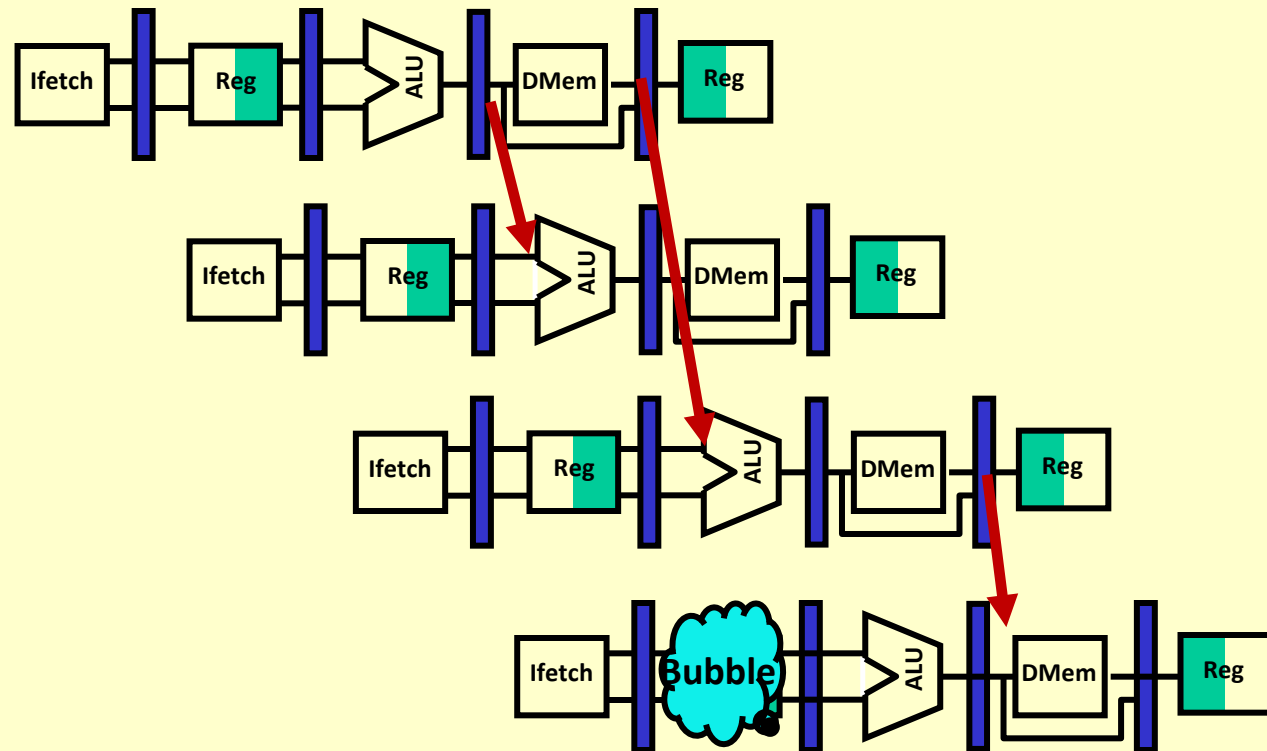
Instruction Order

add r1, r2, r3

sub r4, **r1**, r5

and r6, **r1**, r7

lw r8, **4(r1)**



How to decide

- Keep track of three previous instructions
- If operand is *not* written by any of three previous instructions ...
 - ... then read from register
- If operand is output of ALU operation of *previous* instruction ...
 - ... then forward from ALU stage
- If operand is output from ALU or MEM of 2nd previous instruction ...
 - ... then forward from MEM stage
- Else stall one cycle

This entire decision tree can be programmed into gates in hardware

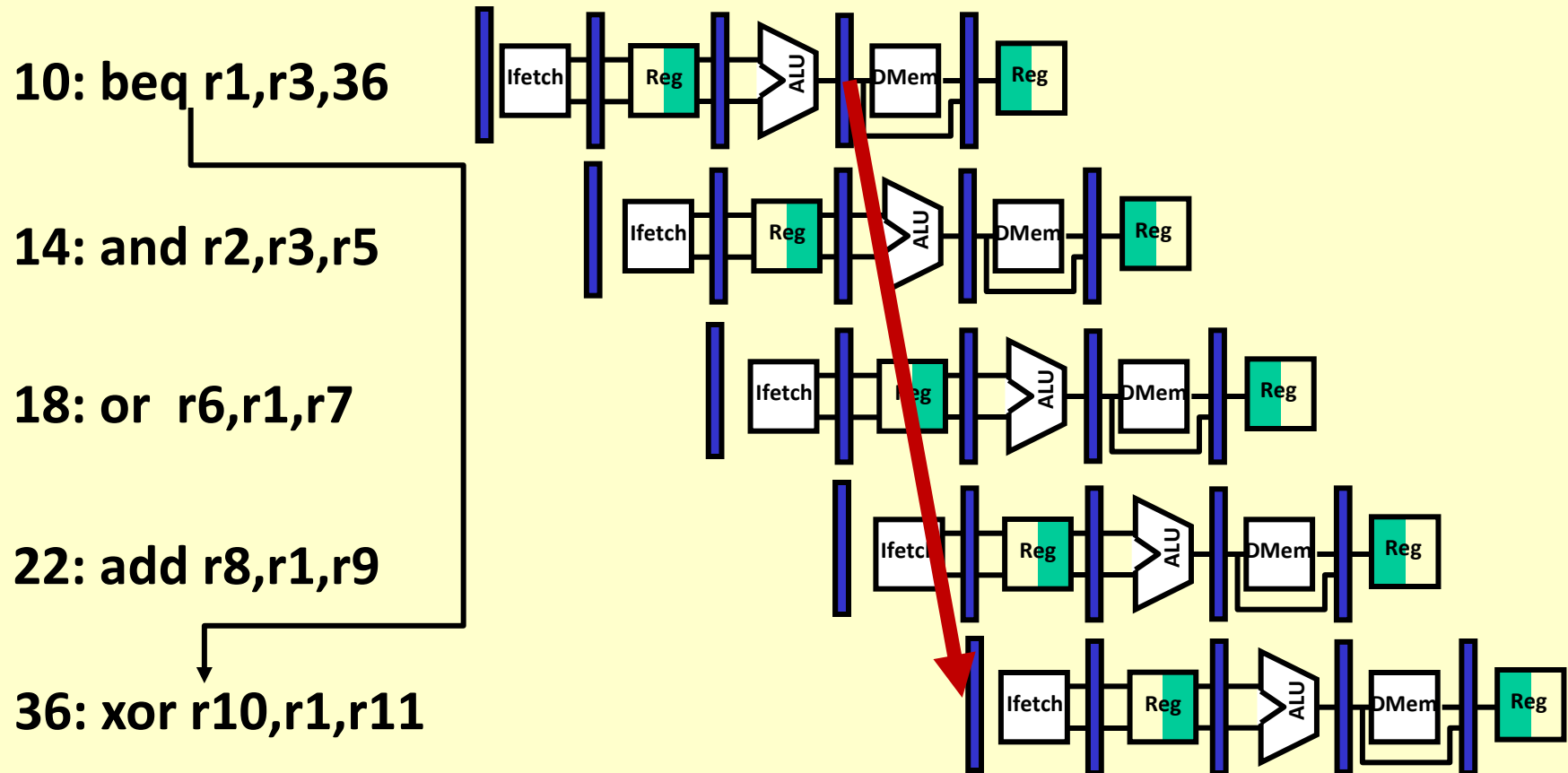
Questions?

Control Hazards

- Decision to take a *branch* or not may *not be known* until after the next instructions are fetched!

Control Hazard on Branches

(Three Stage Stall)



What do you do with the 3 instructions in between?

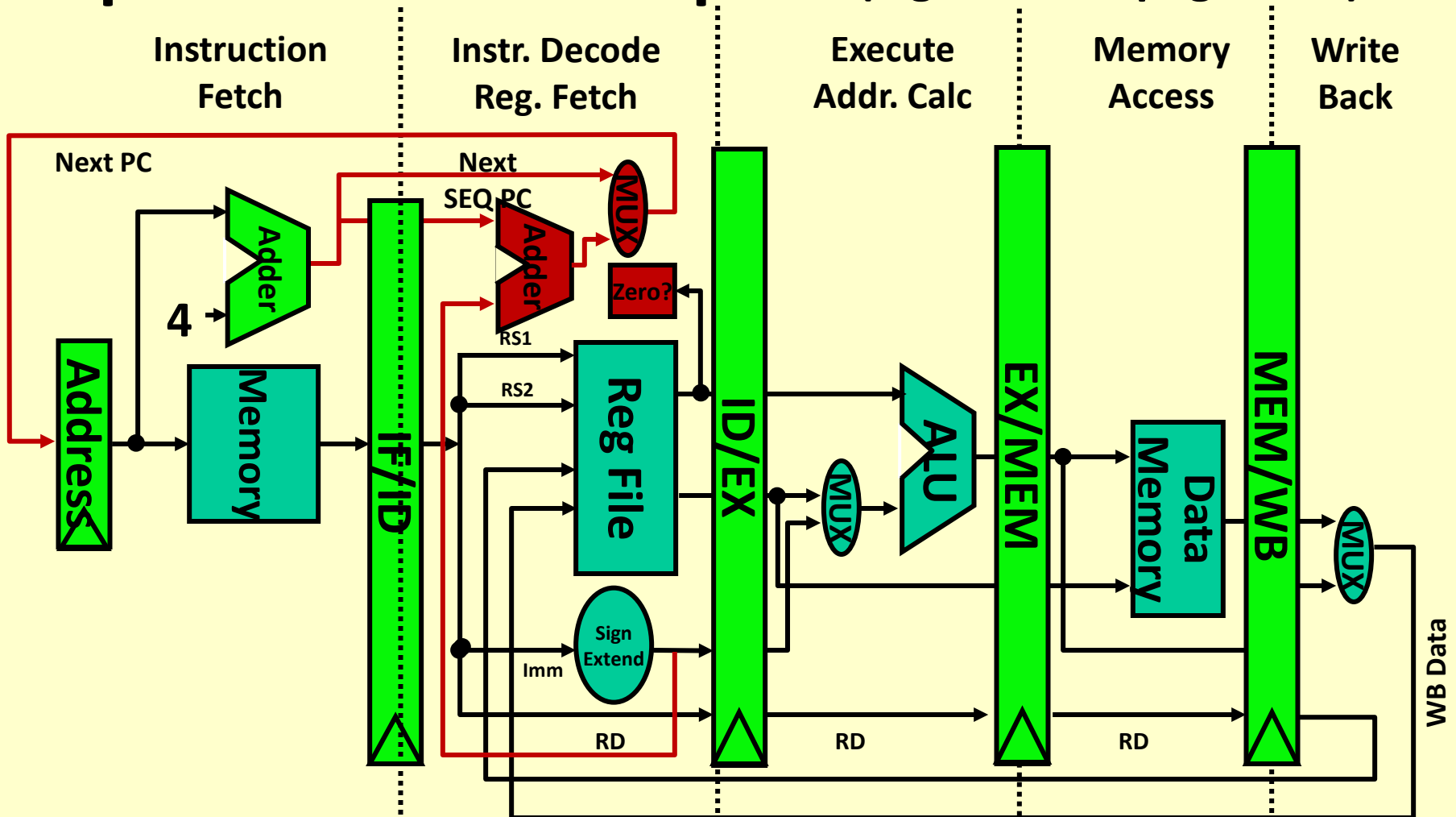
How do you do it?

Where is the “commit”?

Branch Stall Impact

- If $CPI = 1$, 30% branch,
Stall 3 cycles $\Rightarrow$ new $CPI = 1.6!$
- Two part solution:—
 - Determine branch taken or not sooner, AND
 - Compute taken branch address earlier
- MIPS branch tests if register = 0 or $\neq 0$
- MIPS Solution:—
 - Move Zero test to ID/RF stage
 - Adder to calculate new PC in ID/RF stage
 - 1 clock cycle penalty for branch versus 3

Pipelined MIPS Datapath (Figure C.28, page C-42)



- Interplay of instruction set design and cycle time.

Four Branch Hazard Alternatives

- **#1: Stall until branch direction is clear**
- **#2: Predict Branch Not Taken**
 - Execute successor instructions in sequence
 - “Squash” instructions in pipeline if branch actually taken
 - Advantage of late pipeline state update
 - 47% MIPS branches not taken on average
 - PC+4 already calculated, so use it to get next instruction
- **#3: Predict Branch Taken**
 - 53% MIPS branches taken on average
 - **But haven’t calculated branch target address in MIPS**
 - MIPS still incurs 1 cycle branch penalty
 - Other machines: branch target known before outcome
- **#4: (next slide)**

**Speculative
Execution**

Four Branch Hazard Alternatives

#4: Delayed Branch

- Define branch to take place **AFTER** a following instruction

branch instruction

sequential successor₁

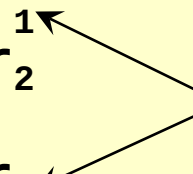
sequential successor₂

.....

sequential successor_n

branch target if taken

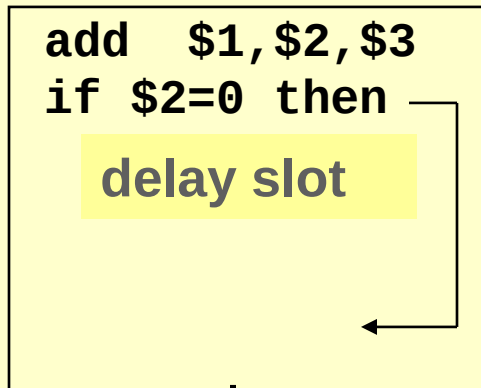
Branch delay of length n



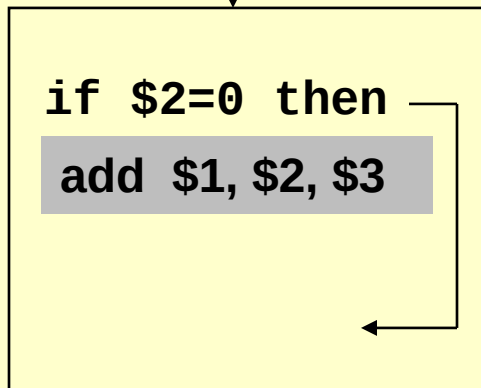
- 1 slot delay allows proper decision and branch target address in 5 stage pipeline
- MIPS uses this

Scheduling Branch Delay Slots (Fig C.14)

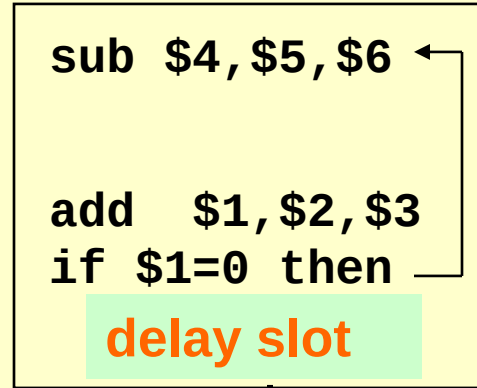
A. From before branch



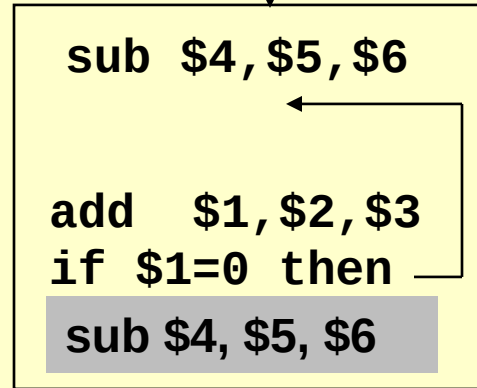
becomes



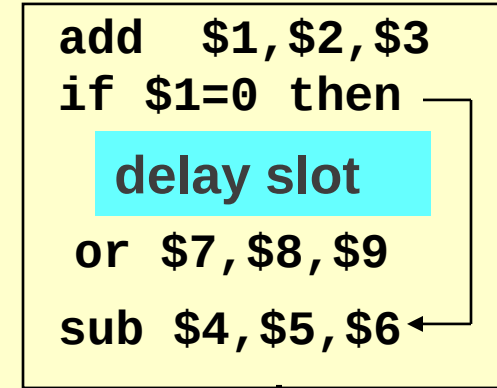
B. From branch target



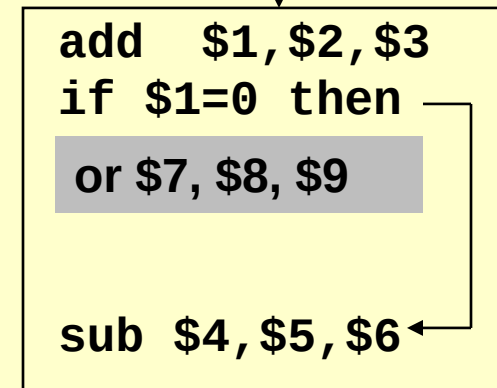
becomes



C. From fall through



becomes



- A is the best choice, fills delay slot & reduces instruction count (IC)
- In B, the sub instruction may need to be copied, increasing IC
- In B and C, must be okay to execute instruction in delay slot whether or not the branch succeeds

Delayed Branch

- **Compiler effectiveness for single branch delay slot:**
 - Fills about 60% of branch delay slots
 - About 80% of instructions executed in branch delay slots useful in computation
 - About 50% ($60\% \times 80\%$) of slots usefully filled
- ...

Delayed Branch — Downside

- **As processors go to deeper pipelines and multiple issue, branch delay grows and need more than one delay slot**
 - Delayed branching has lost popularity compared to more expensive but more flexible dynamic approaches
 - Growth in available transistors has made dynamic approaches relatively cheaper

Evaluating Branch Alternatives

$$\text{SpeedUp} = \frac{\text{PipelineDepth}}{1 + \text{BranchFreq} \times \text{BranchPenalty}}$$

Assume 4% unconditional branch, 6% conditional branch-untaken, 10% conditional branch-taken. Total 20%

<i>Scheduling scheme</i>	<i>Branch penalty</i>	<i>CPI</i>	<i>speedup v. unpipelined</i>	<i>speedup v. stall</i>
Stall pipeline	3	1.60	3.1	1.0
Predict taken	1	1.20	4.2	1.33
Predict not taken	1	1.14	4.4	1.40
Delayed branch	0.5	1.10	4.5	1.45

See also Figure C.16

Questions?

Branch Prediction

- Necessary to improve performance in a pipeline.
- Can be done by the compiler or the hardware.
- *Static* prediction can be used to assist dynamic predictions.

Static Branch Prediction

- Simplest *static* prediction method is to assume a branch is taken
 - Helps to compile more efficient code
- Does not rely on information about executing code.
- Predicts always same direction for a branch during entire program execution
- High and variable misprediction rate

Static Branch Prediction (continued)

- **Alternatively, base on information from earlier runs**
 - Much more accurate
 - Reason: individual branches are highly biased one way or the other

- **Effectiveness depends on the accuracy and frequency of conditional branches.**

Mis-prediction rate (profile-based predictor)

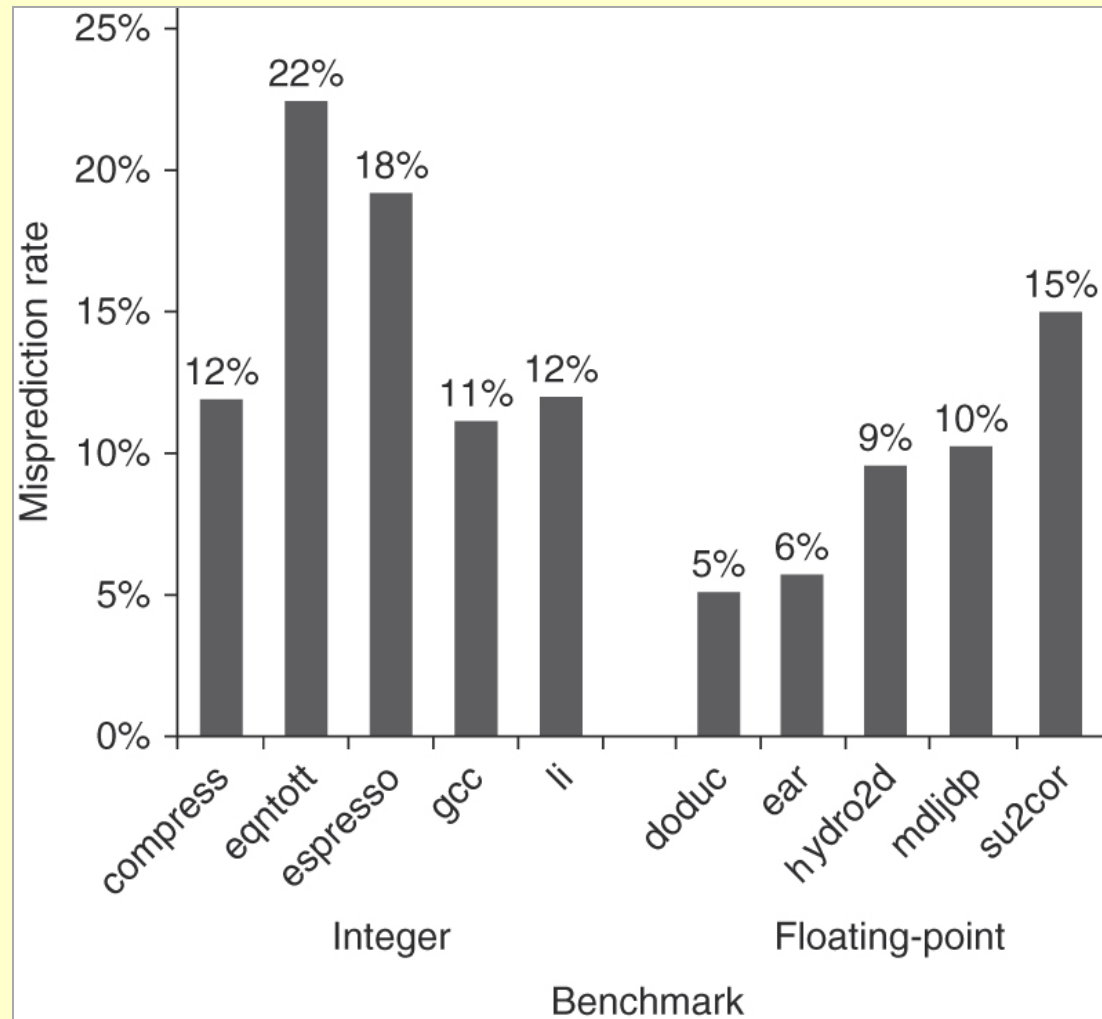


Figure C.17



Static Branch Prediction (continued)

- Alternatively, base on information from earlier runs
 - Much more accurate
 - Reason: individual branches are highly biased one way or the other
- Effectiveness depends on the accuracy and frequency of conditional branches.
- **Rarely used with modern processors.**

Why not?

Branch-prediction buffer

- **Simplest dynamic scheme ‘branch history table’**
 - 1-bit cache of recent branch instructions
- **Instructions in that direction are fetched**
- **If not correct**
 - Invert the bit and store it back into cache
- **Typically mispredicts twice**
 - Not just once!

Branch-Prediction Buffer (continued)

■ Improvement:–

- 2-bit cache entry,
- Prediction must miss twice before change
- See Fig C.18

■ Buffer implemented as a small, special “cache” accessed during the IF pipe stage

- Tag = address of branch instruction

2-bit Branch Prediction

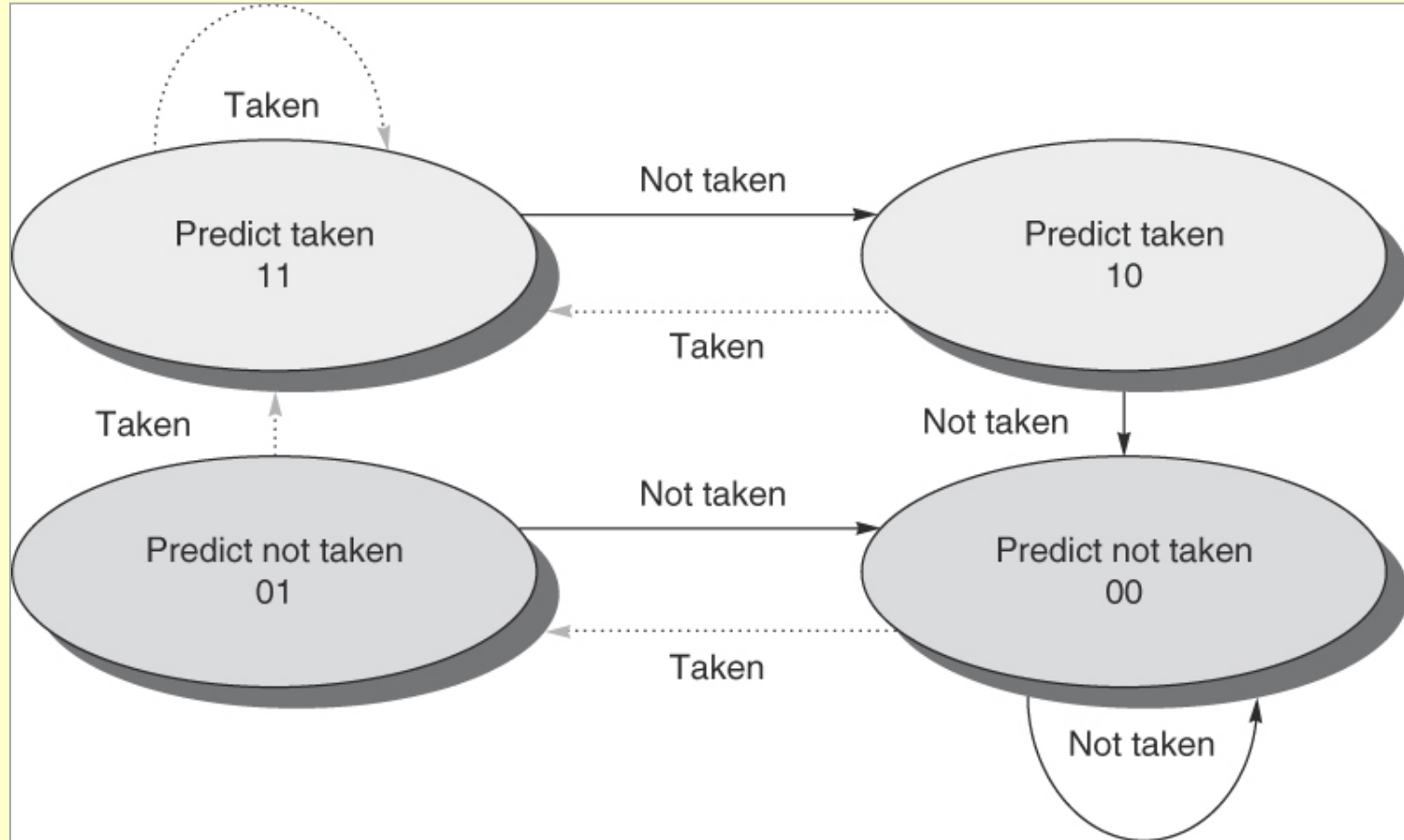


Figure C.18

Example

...

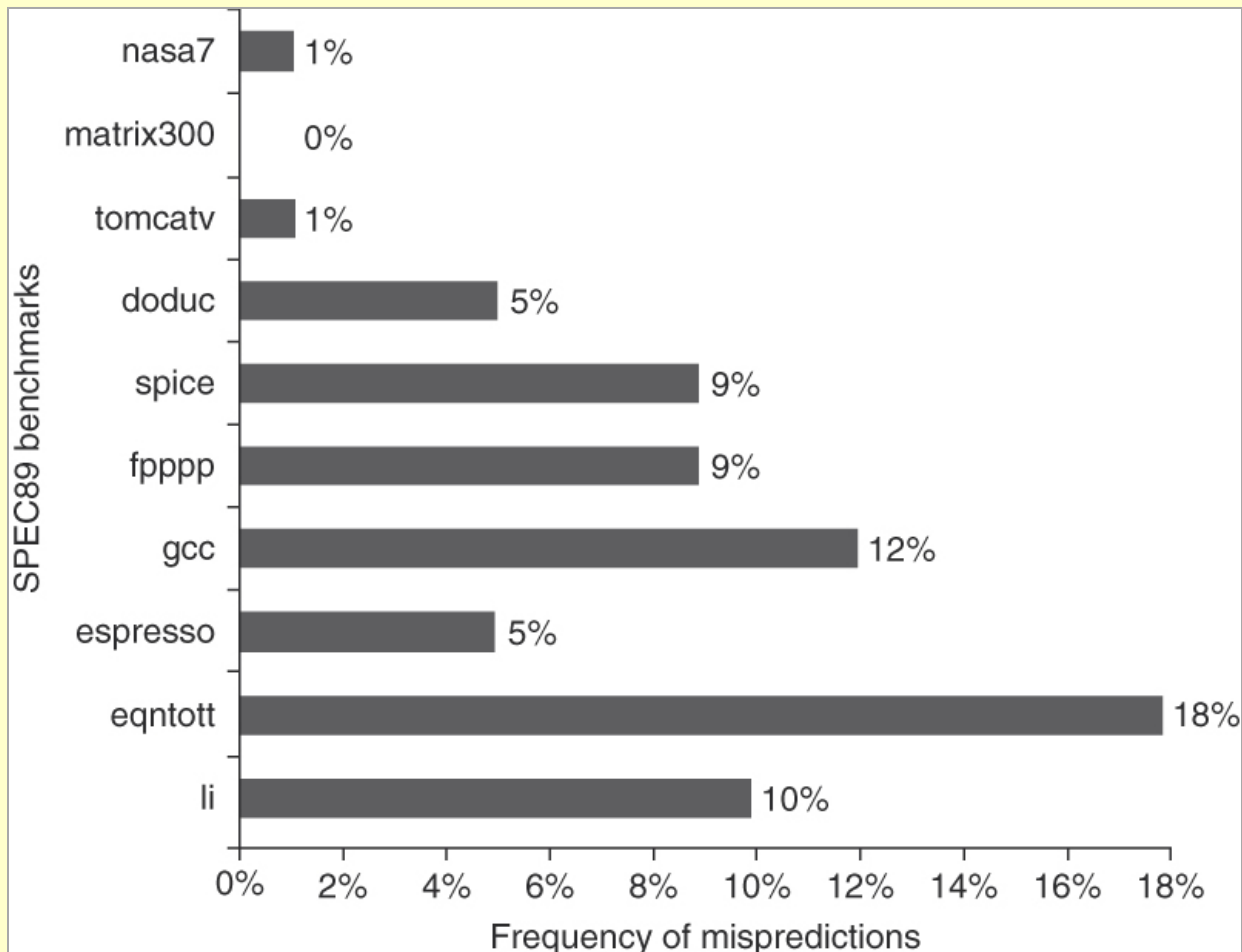
```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        for (int k = 0; k < n; k++)  
            C[i][j] += A[i][k] * B[k][j];
```

...

Branch-prediction buffer (continued)

- **If predicted as ‘taken’, fetching of instructions begins as soon as PC is known**
 - Otherwise, sequential fetching continues.
 - When wrong prediction, bits are changed
- **With 4096 entries, 82%-99% accuracy rate in SPEC89 benchmarks**
 - See Fig C.19

Mis-prediction rates (2-bit prediction buffer)



Correlating predictors

- Increasing number of bits in the predictor => very little improvement
- 2-bit predictor only uses recent information for a single branch.
- Looking at behaviors from other branches can help improve prediction accuracy.

Correlating predictors (continued)

```
if(a==2) {  
    a=0  
}  
if(b==2) {  
    b=0  
}  
if(a!=b) {  
    ...  
}
```

The last branch will only be taken if the first two are not taken as well, otherwise it should always execute.

Using the behavior of a single branch could never capture this behavior

Questions?

More Problems with Pipelining

- **Exception:**— An unusual event happens to an instruction during its execution
 - Examples: divide by zero, undefined opcode
- **Interrupt:**— Hardware signal to switch the processor to a new instruction stream
 - Example: a sound card interrupts when it needs more audio output samples (an audio “click” happens if it is left waiting)
- ...

More Problems with Pipelining (continued)

- **Requirement:– It must appear that the exception or interrupt must appear between 2 instructions (I_i and I_{i+1})**
 - The effect of all instructions up to and including I_i is totally complete
 - No effect of any instruction after I_i can take place
- **Interrupt or exception handler either aborts program or restarts at instruction I_{i+1}**

Questions?